Computational Physics @ NuSTEAM

Lecture 3

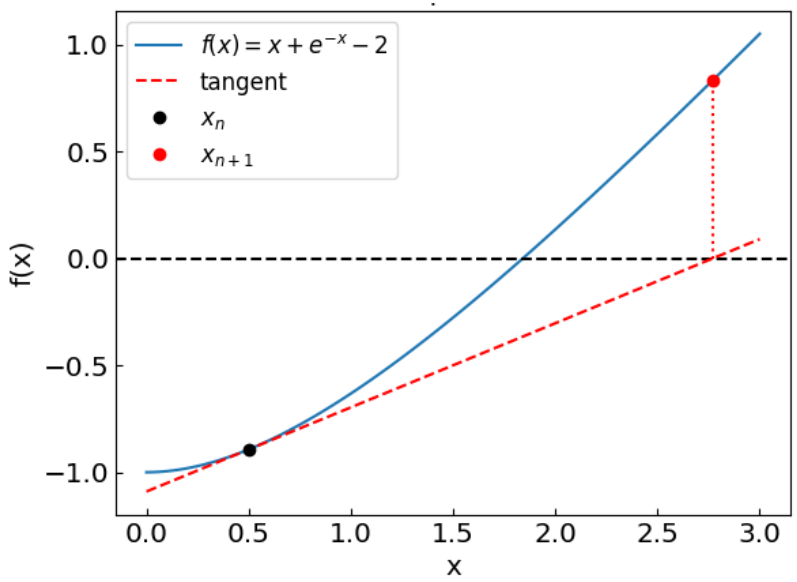
- *Non-linear equations*
- *Random numbers*

June 13, 2025

Instructor: Volodymyr Vovchenko (vvovchenko@uh.edu)

based on Computations Physics course at UH
GitHub: https://github.com/vlvovch/PHYS6350-ComputationalPhysics/tree/nusteam
Online textbook: https://vovchenko.net/computational-physics

Non-linear equations



Jupyter Notebook: 3_NonlinearEquations_NuSTEAM.ipynb

Non-linear equations

Suppose we have an equation $f(x) = 0$

We can evaluate $f(x)$, but we do not know how to solve it for x

Examples:

- Roots of high-order polynomials
- Transcendental equations
- Implicit ODE/PDE methods

Root-finding techniques

Numerical root-finding method: iterative process to determine the root(s) of non-linear equation(s) to desired accuracy

Types:

- Two-point (bracketing)
  - **Bisection method**
  - False position method (regula falsi)

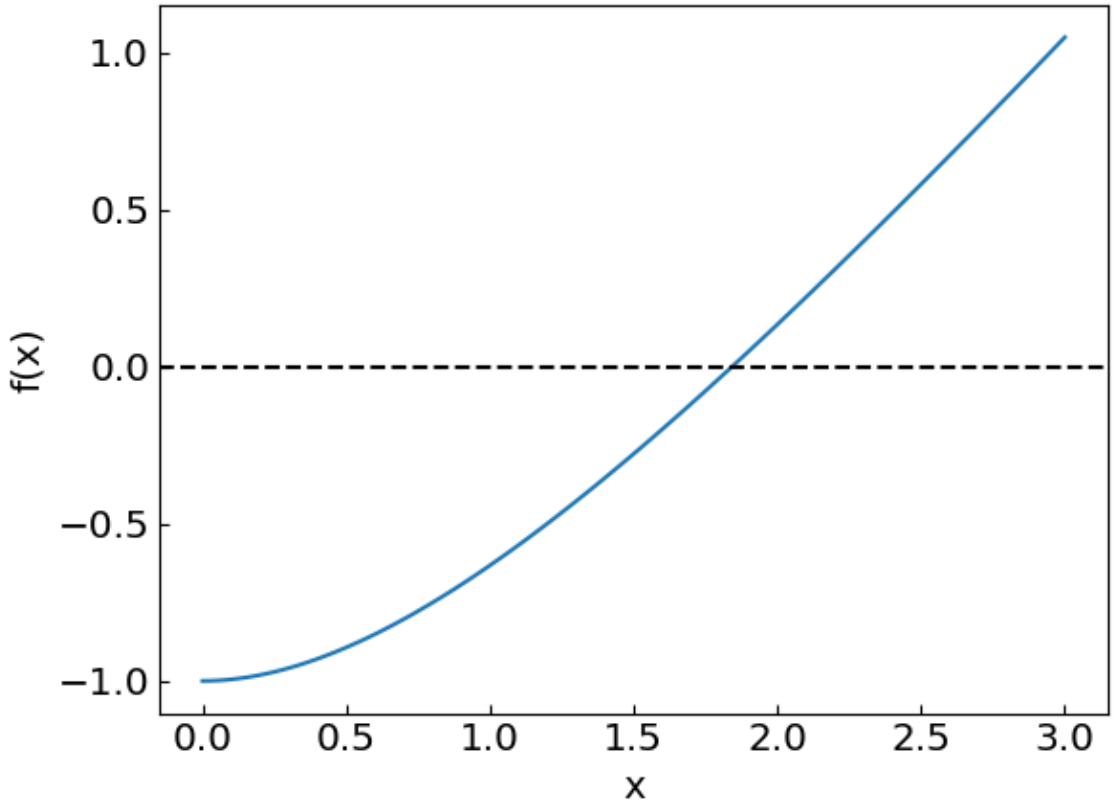
- Local
  - **Secant method**
  - **Newton-Raphson method (using the derivative)**
  - Relaxation method

- Multi-dimensional
  - Newton method
  - Broyden method

Non-linear equations

Consider an equation

$$x + e^{-x} - 2 = 0$$



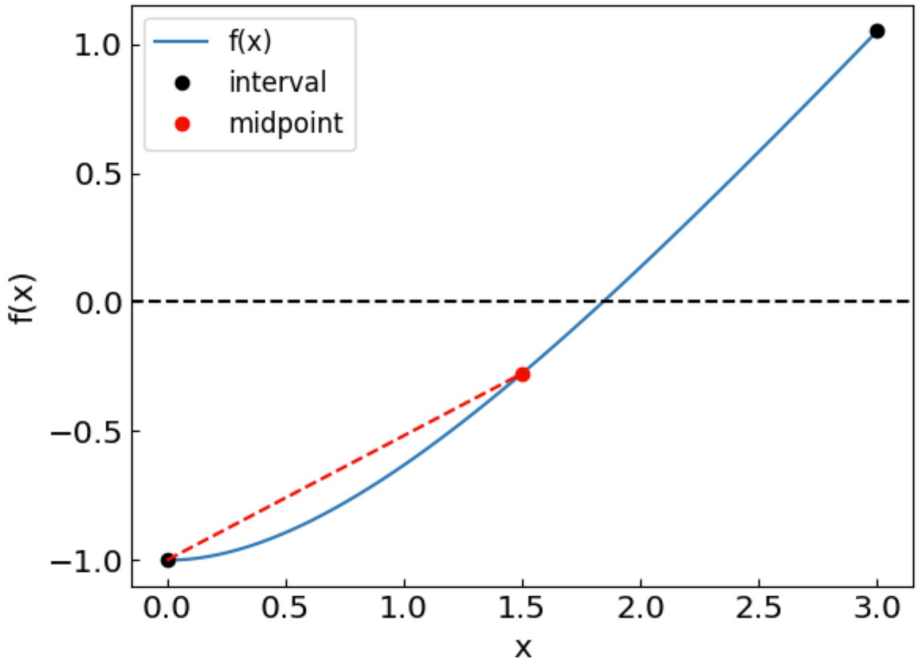
Bisection method

Bisection method:

1. Find an interval (a,b) which brackets the root x*
   - x* ∈ (a,b)
   - f(a) & f(b) have opposite signs

2. Take the midpoint c = (a+b)/2 and halve the interval bracketing the root

3. Repeat the process until the desired precision is achieved

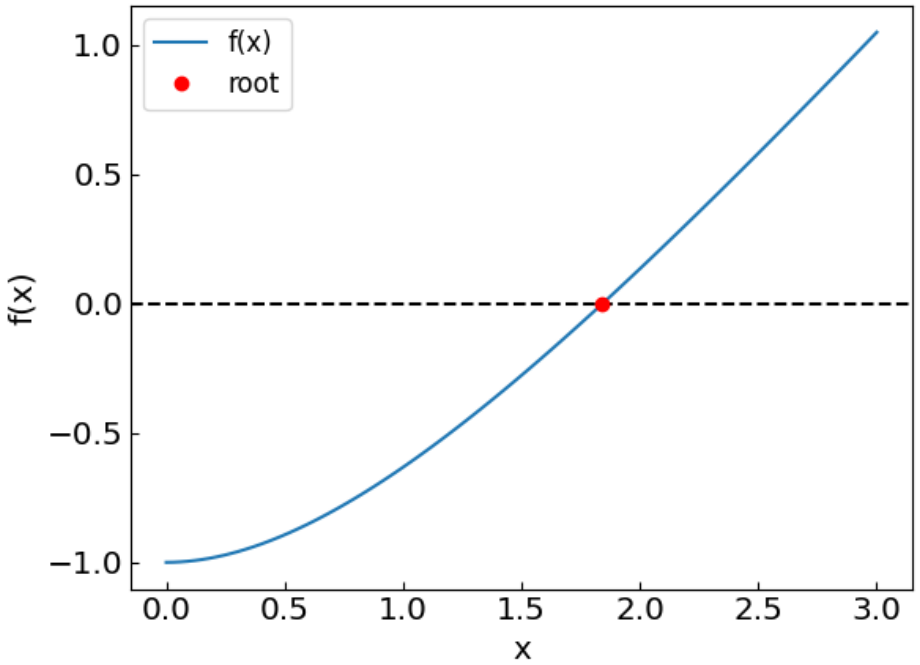


Method is guaranteed to converge to the root
The error is halved at each step – "linear" convergence

Bisection method

```
def bisection_method(
    f,                      # The function whose root we are trying to find
    a,                      # The left boundary
    b,                      # The right boundary
    tolerance = 1.e-10,     # The desired accuracy of the solution
    ):
    fa = f(a)                           # The value of the function at the left boundary
    fb = f(b)                           # The value of the function at the right boundary
    if (fa * fb > 0.):
        return None                     # Bisection method is not applicable

    while ((b-a) > tolerance):
        last_bisection_iterations += 1
        c = (a + b) / 2.                # Take the midpoint
        fc = f(c)                       # Calculate the function at midpoint


        if (fc * fa < 0.):
            b = c                       # The midpoint is the new right boundary
            fb = fc
        else:
            a = c                       # The midpoint is the new left boundary
            fa = fc

    return (a+b) / 2.
```

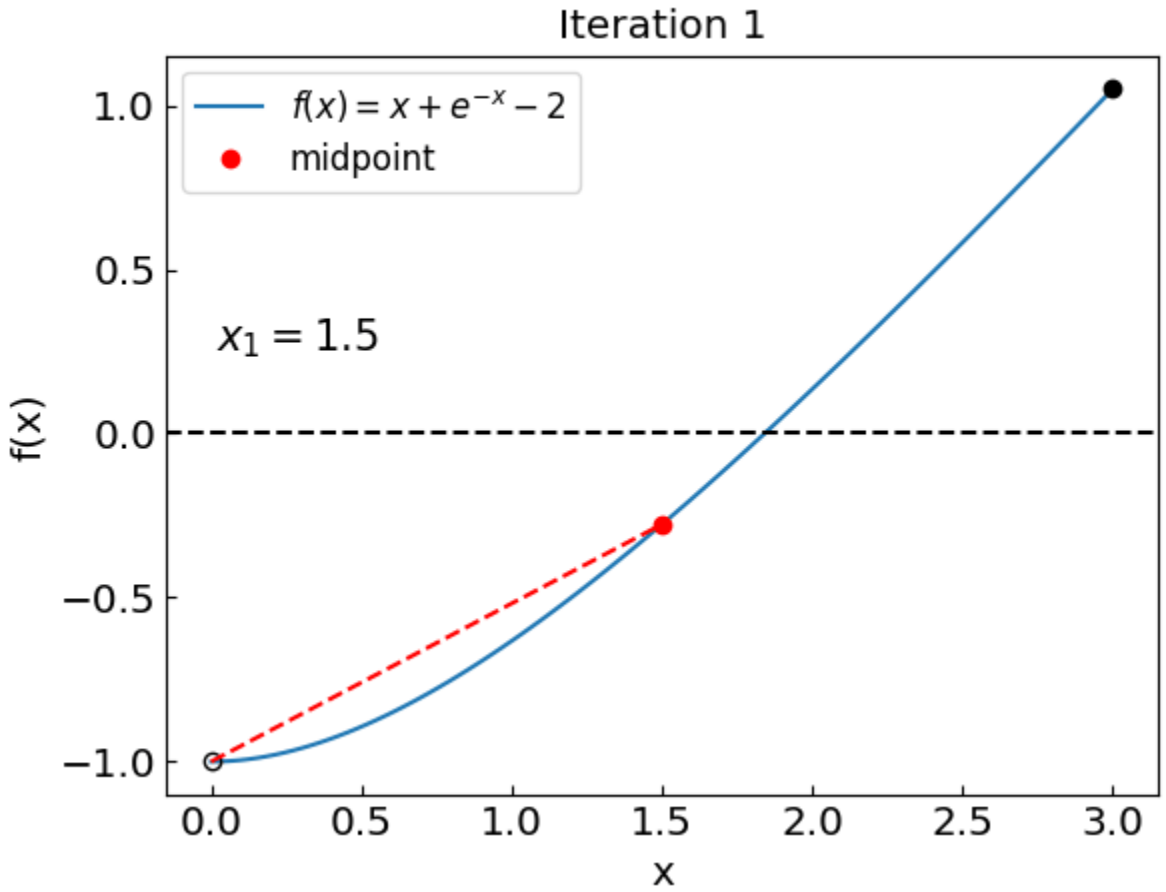
$$x + e^{-x} - 2 = 0$$




```
Solving the equation x + e^-x - 2 = 0 on an interval ( 0.0 , 3.0 ) using bisection method
The solution is x =  1.8414056604233338  obtained with  35  iterations
```

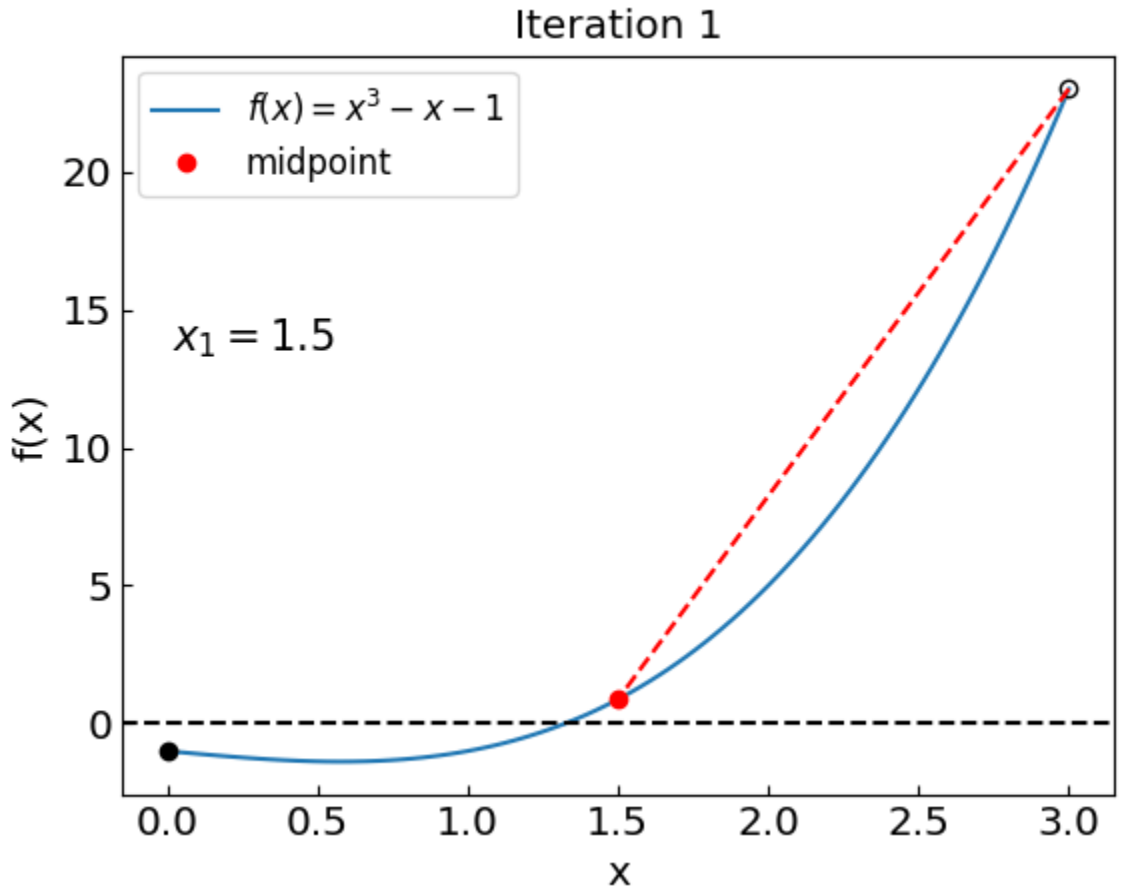
Bisection method: how the iterations look like

$$x + e^{-x} - 2 = 0$$



Bisection method: another example

Let us consider another equation: $x^3 - x - 1 = 0$



35 iterations in both cases

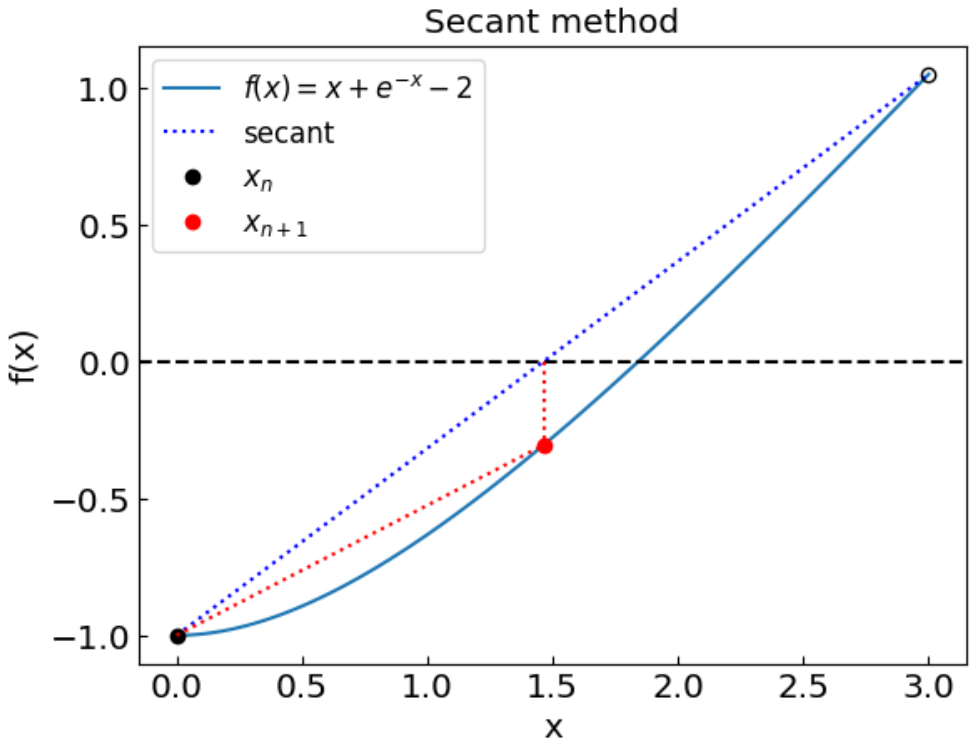
Secant method

Secant method:

1. Find an interval (x_0,x_1), the interval *need not bracket the root*
2. Instead of midpoint take a point where the straight line between the endpoints crosses the y = 0 axis

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

3. Repeat the process until the desired precision $|x_{n+1}-x_n| <$ eps

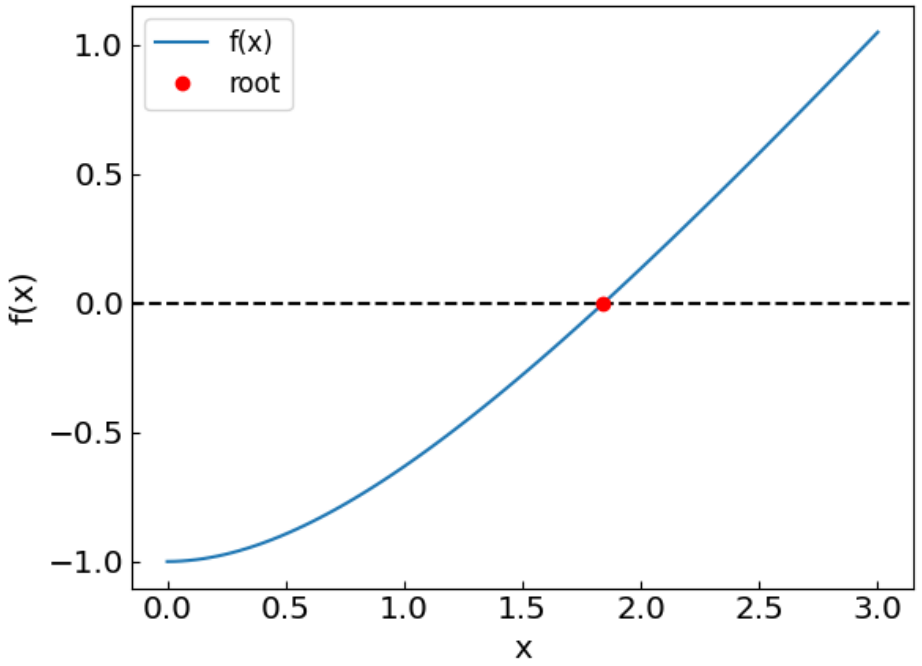


Typically much faster convergence than bisection occurs when the algorithm is effective. However, it can still be slower than bisection or may not converge at all (e.g., the secant may be parallel to the x-axis).

Secant method

```python
def secant_method(
    f,                      # The function whose root we are trying to find
    a,                      # The left boundary
    b,                      # The right boundary
    tolerance = 1.e-10,     # The desired accuracy of the solution
    max_iterations = 100  # Maximum number of iterations
    ):
    fa = f(a)                           # The value of the function at the left boundary
    fb = f(b)                           # The value of the function at the right boundary

    xprev = xnew = a                            # Estimate of the solution from the previous step

    global last_secant_iterations
    last_secant_iterations = 0

    for i in range(max_iterations):
        last_secant_iterations += 1

        xprev = xnew
        xnew = a - fa * (b - a) / (fb - fa) # Take the point where straight line between a and b crosses y = 0
        fnew = f(xnew)                      # Calculate the function at midpoint

        b = a
        fb = fa
        a = xnew
        fa = fnew

        if (abs(xnew-xprev) < tolerance):
            return xnew

    print("Secant method failed to converge to a required precision in " + str(max_iterations) + " iterations")
    print("The error estimate is ", abs(xnew - xprev))

    return xnew
```

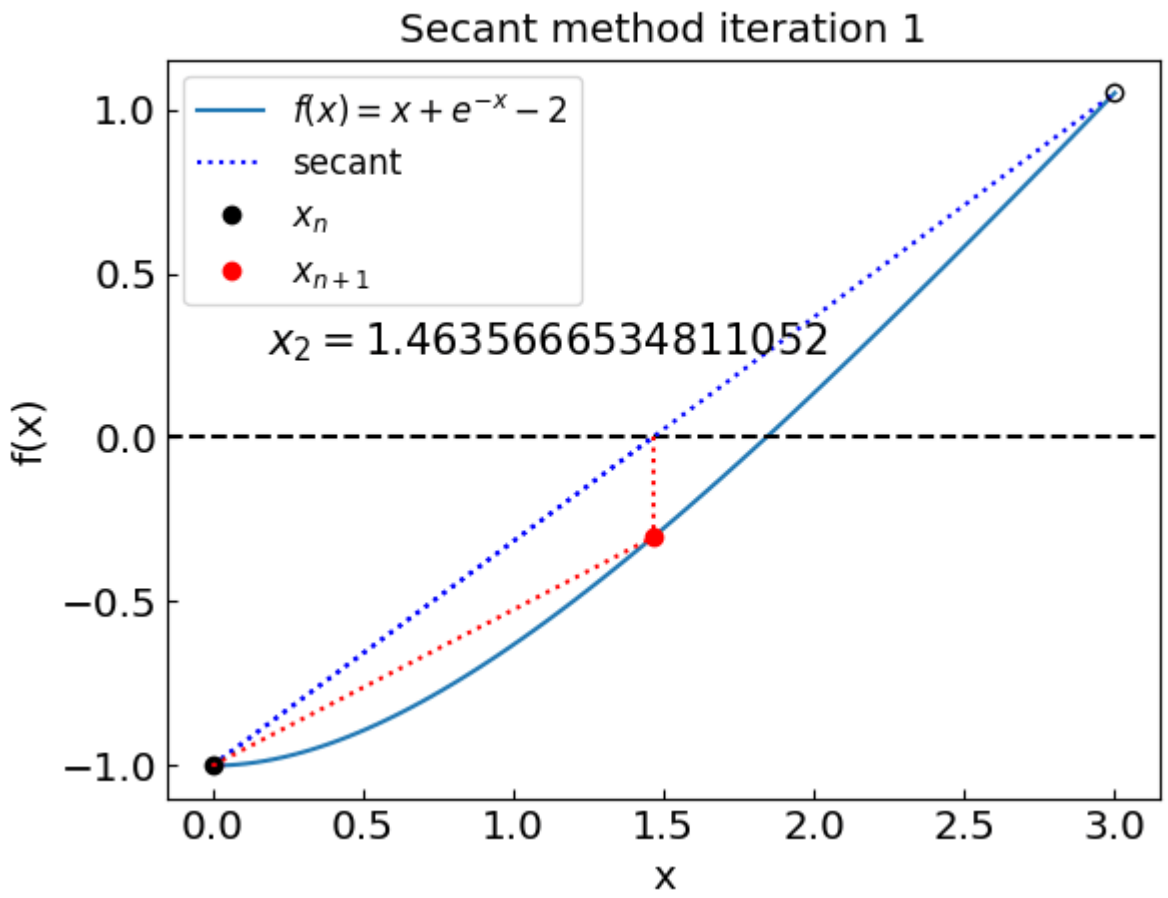
$$x + e^{-x} - 2 = 0$$




```
Solving the equation x + e^-x - 2 = 0 on an interval ( 0.0 , 3.0 ) using the secant method
The solution is x =  1.8414056604369606 obtained after  7  iterations
```

Secant method

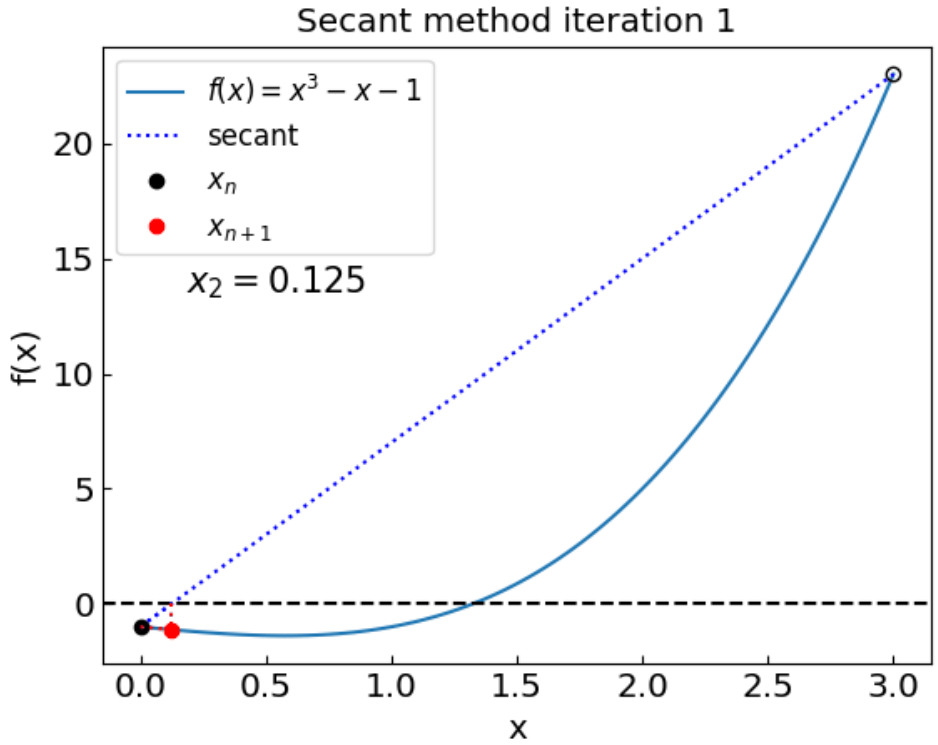
$$x + e^{-x} - 2 = 0$$



Secant method

$$x^3 - x - 1 = 0$$

```
Iteration:     1, x =    0.125000000000000
Iteration:     2, x =   -1.015873015873016
Iteration:     3, x =  -14.026092564115256
Iteration:     4, x =   -1.010979901305751
Iteration:     5, x =   -1.006133240911884
Iteration:     6, x =   -0.512666258317272
Iteration:     7, x =    0.273834681149844
Iteration:     8, x =   -1.287767830907429
Iteration:     9, x =    3.565966235528240
Iteration:    10, x =   -1.077368321415013
Iteration:    11, x =   -0.947522156044583
Iteration:    12, x =   -0.513174359589628
Iteration:    13, x =    0.447558454314033
Iteration:    14, x =   -1.325124217388110
Iteration:    15, x =    4.186373891812861
Iteration:    16, x =   -1.167930924631363
Iteration:    17, x =   -1.058303471905222
Iteration:    18, x =   -0.643978481189561
Iteration:    19, x =   -0.131674045244213
Iteration:    20, x =   -1.933586024088406
Iteration:    21, x =    0.157497929951306
Iteration:    22, x =    0.626623389695762
Iteration:    23, x =   -2.226715128003442
Iteration:    24, x =    1.093727500240917
Iteration:    25, x =    1.382563036703896
Iteration:    26, x =    1.310687668369503
Iteration:    27, x =    1.323983763313963
Iteration:    28, x =    1.324727653842468
Iteration:    29, x =    1.324717950607204
Iteration:    30, x =    1.324717957244686
Iteration:    31, x =    1.324717957244746
```

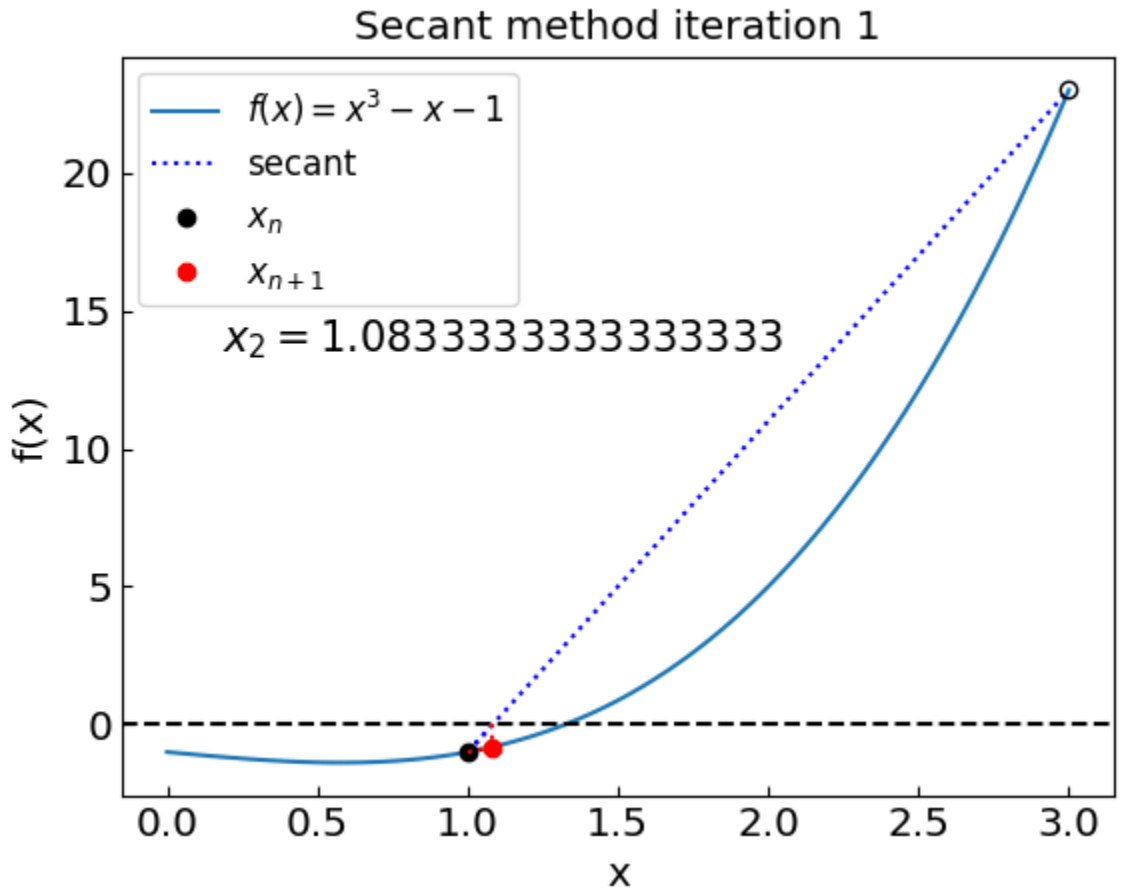


Because the method does not bracket the root, it is not guaranteed to converge
In this case, it managed to recover

Secant method

$$x^3 - x - 1 = 0$$

Choose the initial interval as (1,3) instead of (0,3)



Newton-Raphson method

Newton-Raphson method:

- Local method (uses only the current estimate to get the next one)
- Requires the evaluation of derivative

Idea: Assume that a given point x is close to the root x* [f(x*)=0]

Then

$$f(x^*) \approx f(x) + f'(x)(x^* - x)$$

and since f(x*) = 0 we have

$$x^* \approx x - \frac{f(x)}{f'(x)}$$

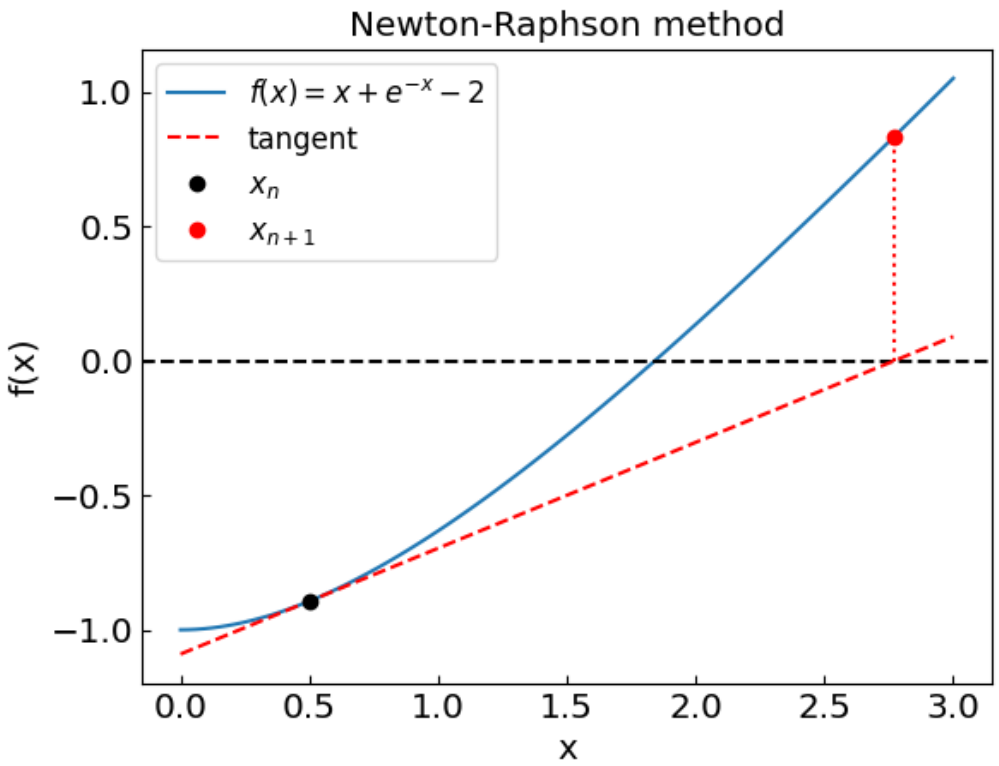
Iterative procedure:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

starting from an initial guess x_0

Newton-Raphson method

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



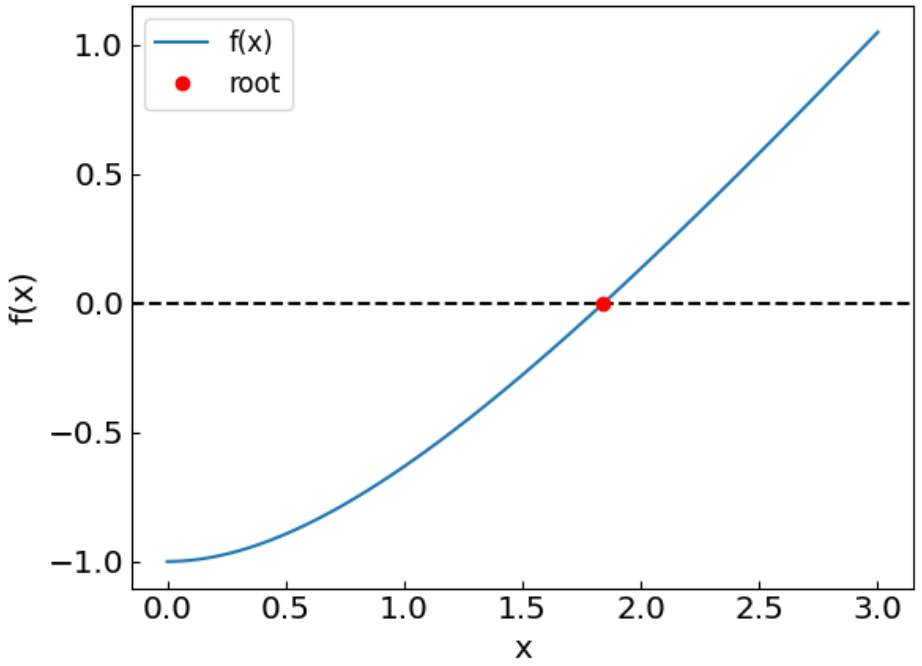
"Quadratic" convergence when works

However, when we are close to f' = 0, we have a problem

Newton-Raphson method

```python
def newton_method(
    f,                    # The function whose root we are trying to find
    df,                   # The derivative of the function
    x0,                   # The initial guess
    tolerance = 1.e-10,   # The desired accuracy of the solution
    max_iterations = 100  # Maximum number of iterations
    ):

    xprev = xnew = x0

    global last_newton_iterations
    last_newton_iterations = 0
    diff = 0.

    for i in range(max_iterations):
        last_newton_iterations += 1

        xprev = xnew
        fval  = f(xprev)                    # The current function value
        dfval = df(xprev)                   # The current function derivative value

        xnew = xprev - fval / dfval         # The next iteration

        if (abs(xnew-xprev) < tolerance):
            return xnew


    print("Newton-Raphson method failed to converge to a required precision in " + str(max_iterations) + " iterations")
    print("The error estimate is ", abs(xnew-xprev))

    return xnew
```

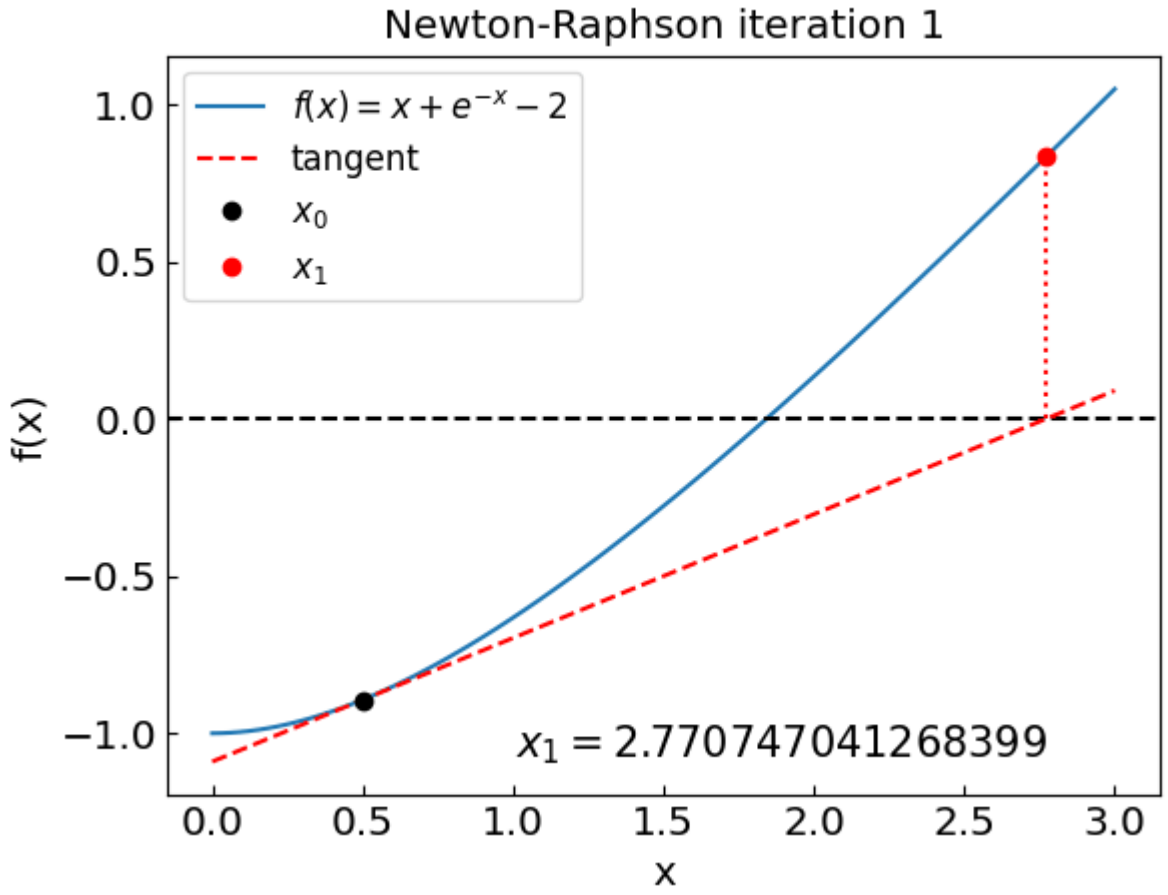
$$x + e^{-x} - 2 = 0$$




```
Solving the equation x + e^-x - 2 = 0 with an initial guess of x0 =  0.5
The solution is x =  1.8414056604369606 obtained after  6  iterations
```

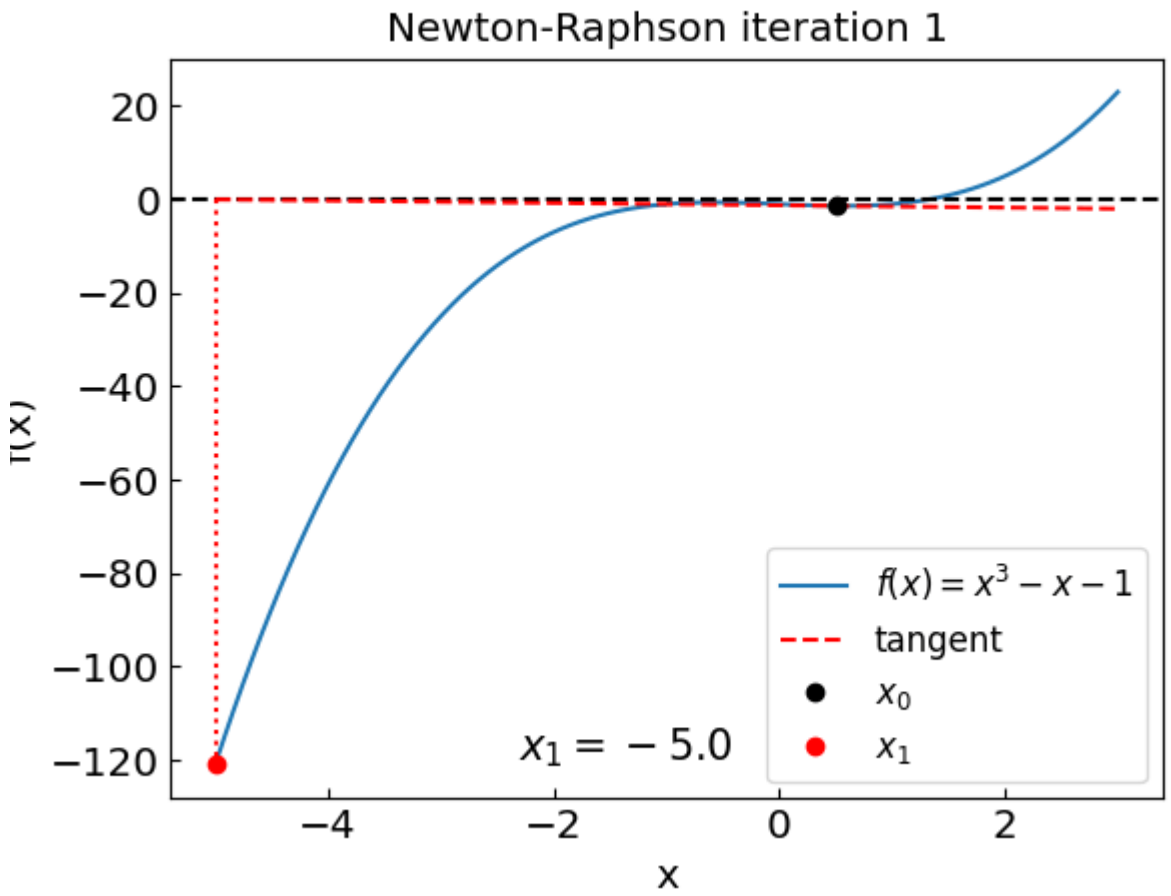
Newton-Raphson method

$$x + e^{-x} - 2 = 0$$



Newton-Raphson method: issues

$$x^3 - x - 1 = 0$$



Similar issue as with the secant method; the reason: f' = 0 at x = 0.577...

Newton-Raphson method: issues

Try finding the root of $f(x) = x^3 - 2x + 2$ with an initial guess of $x_0 = 0$

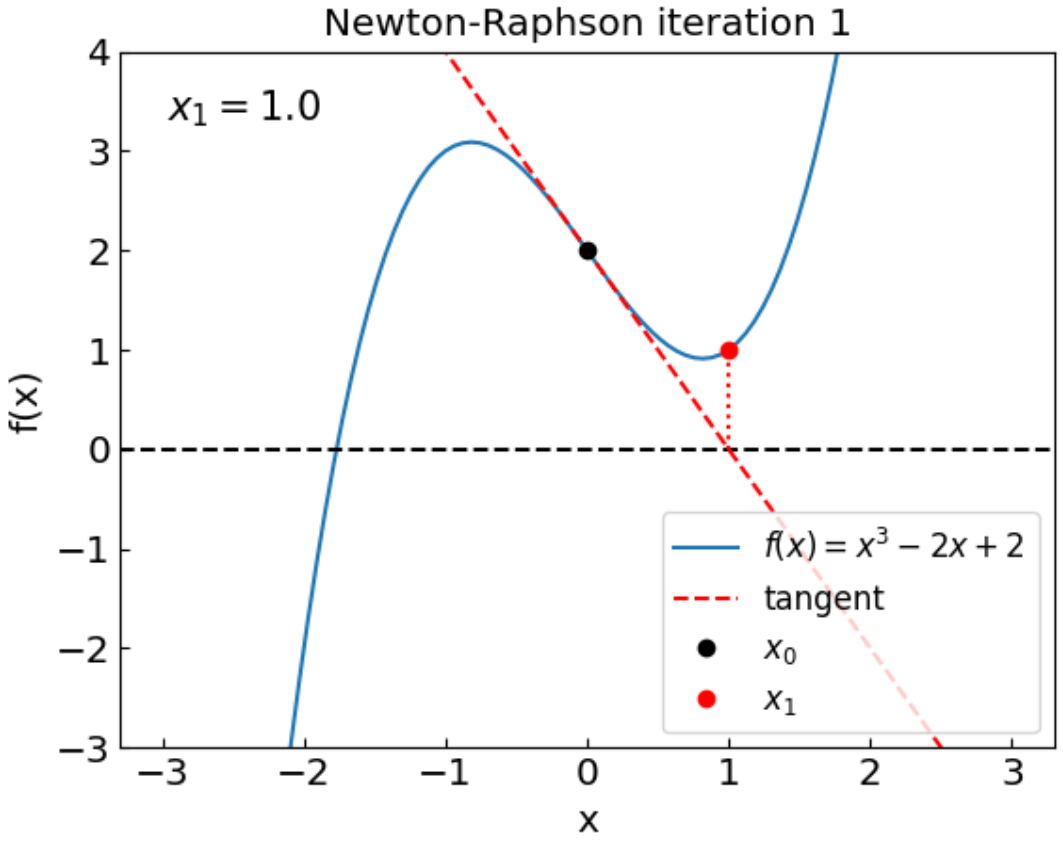
Iteration 1: $f(x_0) = 2, \quad f'(x_0) = -2$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = 1$$

Iteration 2: $f(x_1) = 1 \quad f'(x_1) = 1$

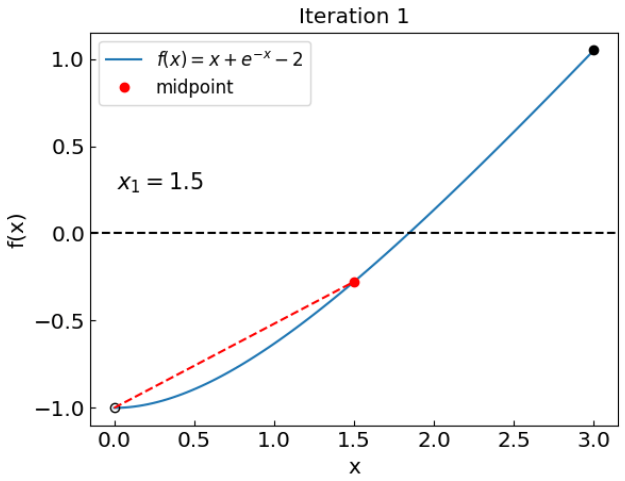
$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = 0$$

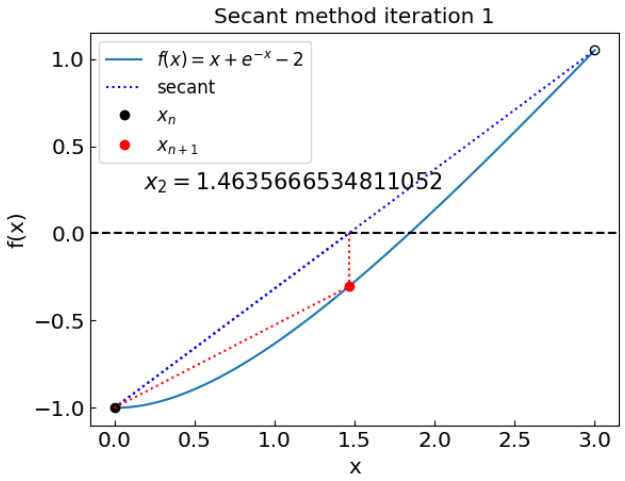
We are back to x_0!

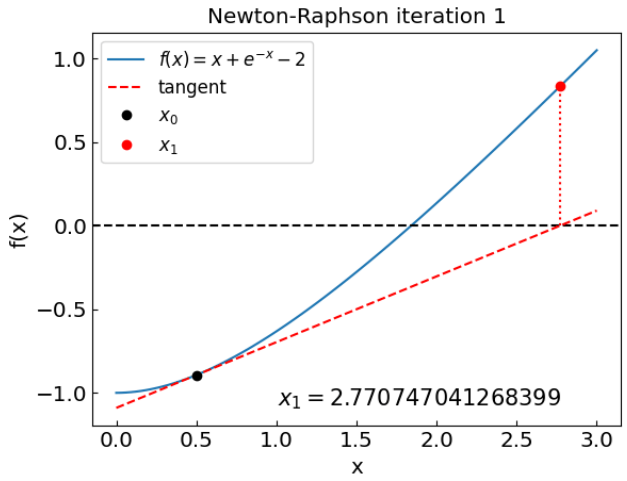


The main issue is, again, we have points with f' = 0 in the neighborhood

Summary


Iteration 1
$f(x) = x + e^{-x} - 2$
midpoint
$x_1 = 1.5$
f(x)
x


Secant method iteration 1
$f(x) = x + e^{-x} - 2$
secant
x_n
x_{n+1}
$x_2 = 1.4635666534811052$
f(x)
x


Newton-Raphson iteration 1
$f(x) = x + e^{-x} - 2$
tangent
x_0
x_1
$x_1 = 2.770747041268399$
f(x)
x

Summary

Bisection method:

- Guaranteed to converge with a fixed rate
- Need to bracket the root

Secant method:

- Typically faster than bisection and false position
- May not always converge

Newton-Raphson method:

- Very fast when converges
- Can be sensitive to initial guess
- May not converge if $f'(x)=0$
- Requires evaluation of the derivative at each step

Systems of non-linear equations

Sometimes we need to solve a system of non-linear equations, e.g.

$$
\begin{aligned}
f_1(x_1, \ldots, x_N) &= 0, \\
f_2(x_1, \ldots, x_N) &= 0, \\
&\ldots \\
f_N(x_1, \ldots, x_N) &= 0
\end{aligned}
$$

Denoting $\mathbf{f} = (f_1, \ldots, f_N)$ and $\mathbf{x} = (x_1, \ldots, x_N)$ this can be written in compact form as

$$\mathbf{f}(\mathbf{x}) = 0\,.$$

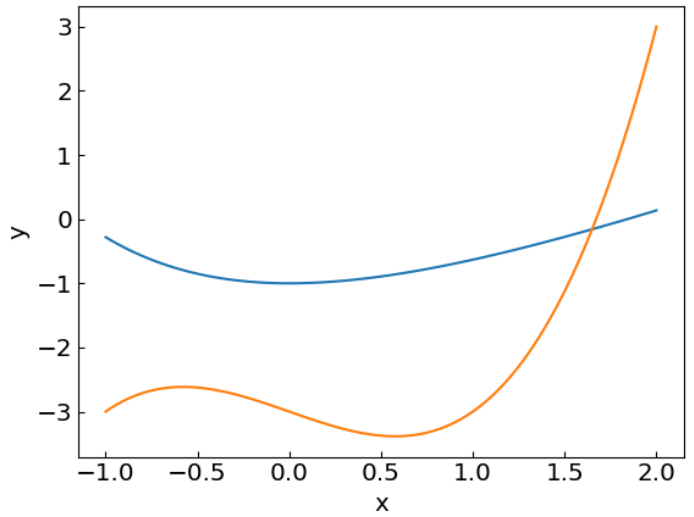
For example:

$$
\begin{aligned}
x + \exp(-x) - 2 - y &= 0, \\
x^3 - x - 3 - y &= 0.
\end{aligned}
$$

i.e.

$$f_1(x_1, x_2) = x_1 + \exp(-x_1) - 2 - x_2$$

$$f_2(x_1, x_2) = x_1^3 - x_1 - 3 - x_2$$



Newton-Raphson method in multiple dimensions

Single equation		System of equations
$f(x) = 0$	Equation(s)	$\mathbf{f}(\mathbf{x}) = 0\,.$
$f(x^*) \approx f(x) + f'(x)(x^* - x)$	Taylor expansion	$\mathbf{f}(\mathbf{x}^*) \approx \mathbf{f}(\mathbf{x}) + J(\mathbf{x})(\mathbf{x}^* - \mathbf{x})$ $J_{ij}(\mathbf{x}) = \frac{\partial f_i}{\partial x_j}$ is Jacobian
$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$	Iteration	$\mathbf{x_{n+1}} = \mathbf{x_n} - J^{-1}(\mathbf{x_n})\,\mathbf{f}(\mathbf{x_n})$

Multi-dimensional Newton-Raphson method requires **matrix inversion** of the Jacobian (J^{-1})

Alternative: Multi-dimensional secant method called **Broyden method**

Random numbers and Monte Carlo methods

Jupyter Notebook: **3_RandomNumbers-NuSTEAM.ipynb**

(Pseudo-)random numbers

Random numbers play important role, both in modelling physics processes (some of which are regarded as truly random, such as radioactive decay) and as a tool to tackle otherwise intractable problems.

Examples:

- Numerical integration (especially in many dimensions)
- Sampling microstates in statistical mechanics
- Simulating quantum processes
- Monte Carlo event generators

Numbers generated on a computer are not truly random, but a good generator produces numbers that reflect the desired properties of a random variable, hence they are called pseudo-random.

Pseudo-random numbers on a computer

- The most basic routine typically produces a random integer number x between 0 and some maximum value m.

- By dividing over m one can get a real pseudo-random number $\eta = x/m$ which is uniformly distributed in an interval $\eta \in (0,1)$

- By applying various transformations and techniques to the sequence of η one can sample various other (non-uniform) distributions.

Most common routine for sampling x is **Mersenne Twister**

Python:

```python
# Use Mersenne Twister
import numpy as np

np.random.rand() # Random number \eta uniformly distributed over (0,1)
```

C++ (since C++11):

avoid using rand()

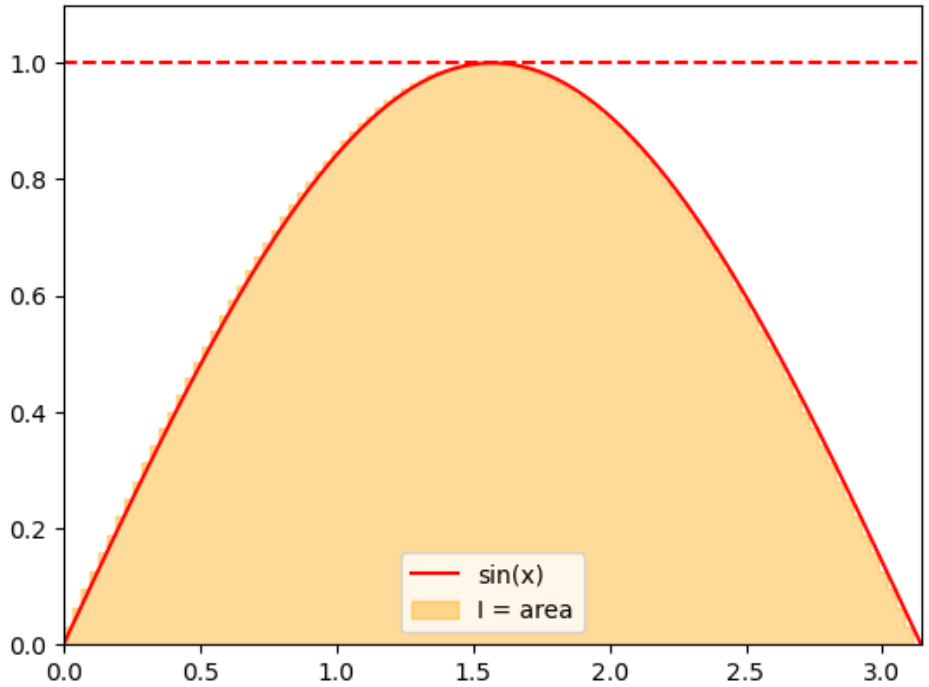
```cpp
#include <ctime>
#include <iostream>
#include <random>
using namespace std;

int main()
{
  // Initializing the sequence
  // with a seed value
  // similar to srand()
  mt19937 mt(time(nullptr));

  // Printing a random number
  // similar to rand()
  cout << mt() << '\n';
  return 0;
}
```

Computing integrals: Estimating the area under the curve

Recall the interpretation of a definite integral as the area under the curve.
We can use this interpretation to apply random numbers for approximating integrals.

Consider

$$I = \int_0^{\pi} \sin(x)dx$$



Computing integrals: Estimating the area under the curve

We can estimate the area by sampling the points uniformly from an enveloping rectangle and counting the fraction of points under the curve given by the integrand f(x).

Assuming an integral

$$I = \int_a^b f(x)dx$$

where f(x) $>$ 0 and f(x) $<$ y_{max}, the integrand can be evaluated as

$$I = (b-a)y_{\max}\frac{C}{N},$$

where C is the number of the sampled points that fall under f(x).

The statistical error of the integrand can be estimated using the properties of the binomial distribution with p = C/N:

$$\delta I = (b-a)y_{\max}\sqrt{\frac{p(1-p)}{N}}$$

The error scales with $N^{-1/2}$

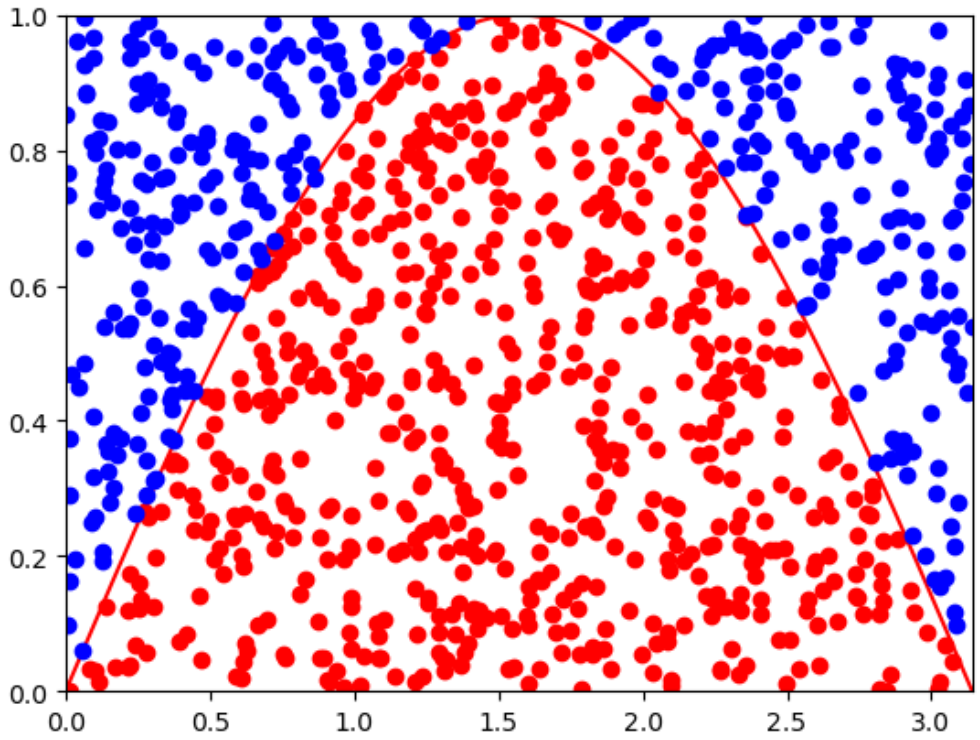
To reduce the error by factor x2 we need to sample x4 more numbers – true for most Monte Carlo methods.

Computing integrals: Estimating the area under the curve

```
# For visualization
points_in = []
points_out = []

# Compute integral \int_a^b f(x) dx as an area below the curve
# Assumes that f(x) is non-negative and bounded from above by ymax
# Returns the value of the integral and the error estimate
def areaMC(f, N, a, b, ymax):
    global points_in, points_out
    points_in = []
    points_out = []
    count = 0
    for i in range(N):
        x = a + (b-a)*np.random.rand()
        y = ymax * np.random.rand()
        if y<f(x):
            count += 1
            points_in.append([x,y])
        else:
            points_out.append([x,y])
    p = count/N
    return (b-a) * ymax * p, (b-a) * ymax * np.sqrt(p*(1-p)/N)
```

```
def f(x):
    return np.sin(x)

N = 1000
I, err = areaMC(f, N, 0, np.pi, 1)
print("I = ",I," +- ",err)
```

```
I =  2.004336112990288  +-  0.04774352682885915
```



Computing π

Consider a circle of unit radius r = 1. Its area is:

$$A = \pi r^2 = \pi$$

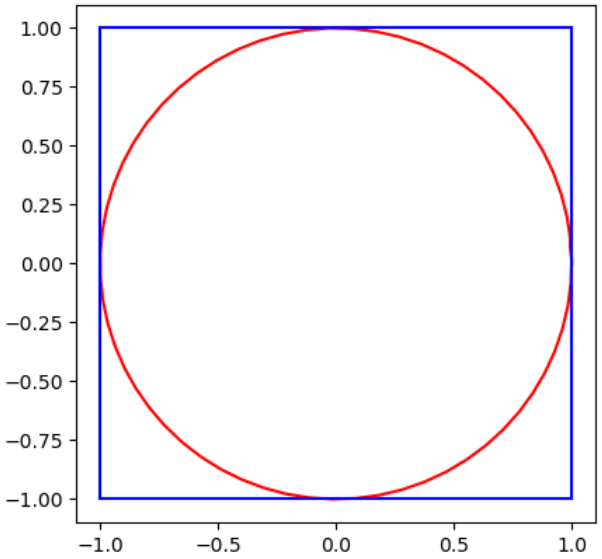
The circle can be embedded into a square with a side length of two. The area of the square is: $A_{sq} = 2^2 = 4$

Consider now a random point anywhere inside the square. The probability that it is also inside the circle is the ratio of their areas:

$$P = \frac{A}{A_{\text{sq}}} = \frac{\pi}{4}.$$

This probability can be estimated by sampling points inside the square many times and counting how many fall inside the circle. π can therefore be estimated as:

$$\pi = 4\frac{A}{A_{\text{sq}}}$$



Computing π

Consider a circle of unit radius r = 1. Its area is:

$$A = \pi r^2 = \pi$$

The circle can be embedded into a square with a side length of two. The area of the square is: $A_{sq} = 2^2 = 4$

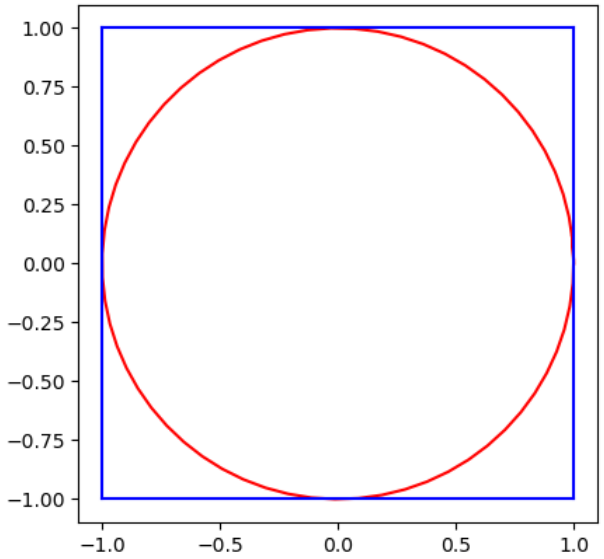
Consider now a random point anywhere inside the square. The probability that it is also inside the circle is the ratio of their areas:

$$P = \frac{A}{A_{\text{sq}}} = \frac{\pi}{4}.$$

This probability can be estimated by sampling points inside the square many times and counting how many fall inside the circle. π can therefore be estimated as:

```
# Compute the value of \pi through the fraction of random points inside a square
# that are also inside a circle around the origin
# Returns the value of the integral and the error estimate
def piMC(N):
    global points_in, points_out
    points_in = []
    points_out = []
    count = 0
    for i in range(N):
        x = -1 + 2 * np.random.rand()
        y = -1 + 2 * np.random.rand()
        r2 = x**2 + y**2
        if (r2 < 1.):
            count += 1
            points_in.append([x,y])
        else:
            points_out.append([x,y])
    p = count/N
    return 4. * p, 4. * np.sqrt(p*(1-p)/N)
```

$$\pi = 4\frac{A}{A_{\text{sq}}}$$



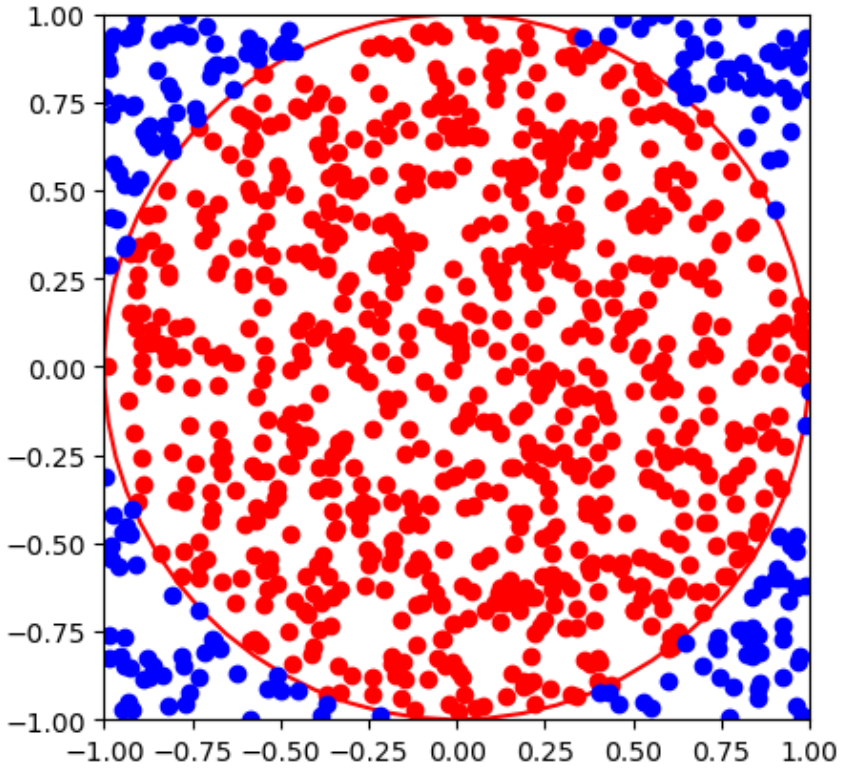
This method is known as the **Monte Carlo estimation of π**.

Computing pi

```
N = 1000
piMC, piMCerr = piMC(N)
print("pi = ",piMC," +- ",piMCerr)
```

```
pi =  3.208  +-  0.050405713961811906
```

Try a larger number of points



Computing integral as the average

- The integral of a function over an interval (a, b) is given by:

$$I = \int_a^b f(x)dx$$

- The mean value of f(x) over (a, b) is:

$$\langle f \rangle = \frac{\int_a^b f(x)dx}{b-a} = \frac{I}{b-a}, \qquad \text{which gives:} \quad I = (b-a)\langle f \rangle$$

- The integral can be estimated by computing the average value of f(x), where x is randomly sampled over (a, b):

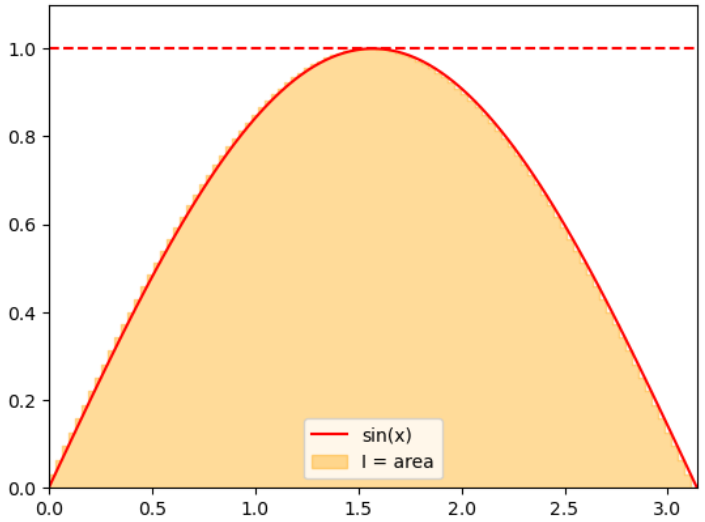
$$\langle f \rangle = \frac{1}{N}\sum_{i=1}^{N} f(x_i).$$

- Using the law of averages, the error estimate involves the variance of f(x):

$$\delta I = (b-a)\sqrt{\frac{\langle f^2 \rangle - \langle f \rangle^2}{N}}$$

```
def intMC(f, N, a, b):
    total = 0
    total_sq = 0
    for i in range(N):
        x = a + (b-a)*np.random.rand()
        fval = f(x)
        total += fval
        total_sq += fval * fval
    f_av = total / N
    fsq_av = total_sq / N
    return (b-a) * f_av, (b-a) * np.sqrt((fsq_av - f_av*f_av)/N)
```

Computing integral as the average




```
def f(x):
    return np.sin(x)

N = 1000
I, err = intMC(f, N, 0, np.pi)
print("I = ",I," +- ",err)
```

```
I =  1.964605422837963  +-  0.030792720278272654
```

Advantages:

- the method works also if $f(x)$ is negative
- no need to know its maximum value

Another way to compute pi

Consider an integral

$$4 \int_0^1 \frac{1}{1+x^2} dx = 4 \arctan(x)|_0^1 = \pi$$

Another way to compute pi

Consider an integral

$$4\int_0^1 \frac{1}{1+x^2}dx = 4\arctan(x)|_0^1 = \pi$$

```
def fpi(x):
    return 4 / (1 + x**2)

N = 10000
I, err = intMC(fpi, N, 0, 1)
print("pi = ",I," +- ",err)
```

```
pi =  3.1365784339451928  +-  0.006449180867490663
```

Computing multi-dimensional integrals

Monte Carlo methods really shine when it comes to numerical evaluation of integrals in multiple dimensions. Consider the following D-dimensional integral

$$I = \int_{a_1}^{b_1} dx_1 \ldots \int_{a_D}^{b_D} dx_D f(x_1, \ldots, x_D).$$

Computing it numerically using for instance the *rectangle rule* would involve the evaluation of a multi-dimensional sum

$$I \approx \sum_{k_1=1}^{N_1} \ldots \sum_{k_D=1}^{N_D} f(x_{k_1}, \ldots, x_{k_D}) \prod_{d=1}^{D} h_d,$$

where $h_d = (b_d - a_d)/N_d$ and $x_{k_d} = a_d + h_d(k_d - 1/2)$.

The total number of integrand evaluations is $N_{tot} = \prod_{d=1}^{N_D} N_d$,
e.g. if we use the same number N of points in each dimension, N_{tot} scales exponentially with D

$$N_{\text{tot}} = N^D$$

curse of dimensionality

Computing multi-dimensional integrals: Monte Carlo

Similar to 1D case, replace

$$I = \int_{a_1}^{b_1} dx_1 \ldots \int_{a_D}^{b_D} dx_D f(x_1, \ldots, x_D).$$

by the mean

$$I = \langle f(x_1, \ldots, x_D) \rangle \prod_{k=1}^{D} (b_k - a_k).$$

Here $x_1,\ldots,x_D$ are independent random variables distributed uniformly in intervals x_k in $[a_k,b_k]$.

Error estimate:

$$\delta I = \sqrt{\frac{\langle f^2 \rangle - \langle f \rangle^2}{N}} \prod_{k=1}^{D} (b_k - a_k),$$

Increasing the number of dimensions by one: sample one more number each iteration.

linear complexity in D

Nonuniformly distributed random numbers

In many cases we deal with random numbers ξ that are distributed non-uniformly.

Common examples are:

- Exponential distribution $\rho(x) = e^{-x}$.
- Gaussian distribution $\rho(x) \propto e^{-\frac{x^2}{2\sigma^2}}$.
- Power-law distribution $\rho(x) \propto x^{\alpha}$.
- Arbitrary peaked distributions.

There are two common methods for generating nonuniform random variates.
They both make use of uniformly distributed variates.

- Inverse transform sampling
- Rejection sampling

Inverse transform sampling

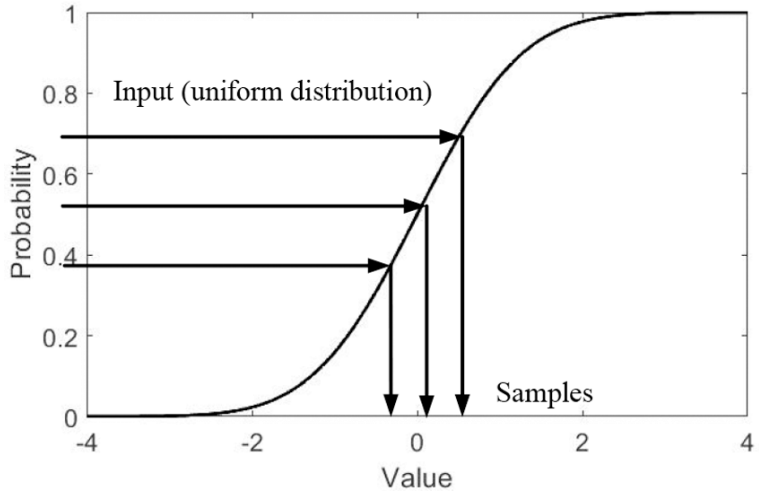
The algorithm follows these steps:

1. Calculate the cumulative distribution function (CDF)

$$G(x) = \int_{-\infty}^{x} \rho(\xi) d\xi$$

2. Find the inverse function $G^{-1}(y)$ as the solution to the equation

$$G(x) = y$$



3. Sample uniformly distributed random variables η and compute ξ using the inverse function:

Main idea: G(x) is monotonic and between 0 and 1, where it grows quicker is where more probability is

Challenges: Sometimes, evaluating G(x) and/or $G^{-1}(y)$ explicitly is difficult. In such cases, numerical integration and/or non-linear equation solvers may be required.

Inverse transform sampling: Exponential distribution

Example: Exponential Distribution

1. Recall the radioactive decay process. The time of decay is distributed according to the probability density function:

$$\rho(t) = \frac{1}{\tau} e^{-\frac{t}{\tau}}.$$

2. The cumulative distribution function is given by:

$$F(x) = \int_0^x \frac{1}{\tau} e^{-\frac{t}{\tau}} dt = 1 - e^{-\frac{x}{\tau}}.$$

3. To apply inverse transform sampling, we need to invert F(x) by solving:

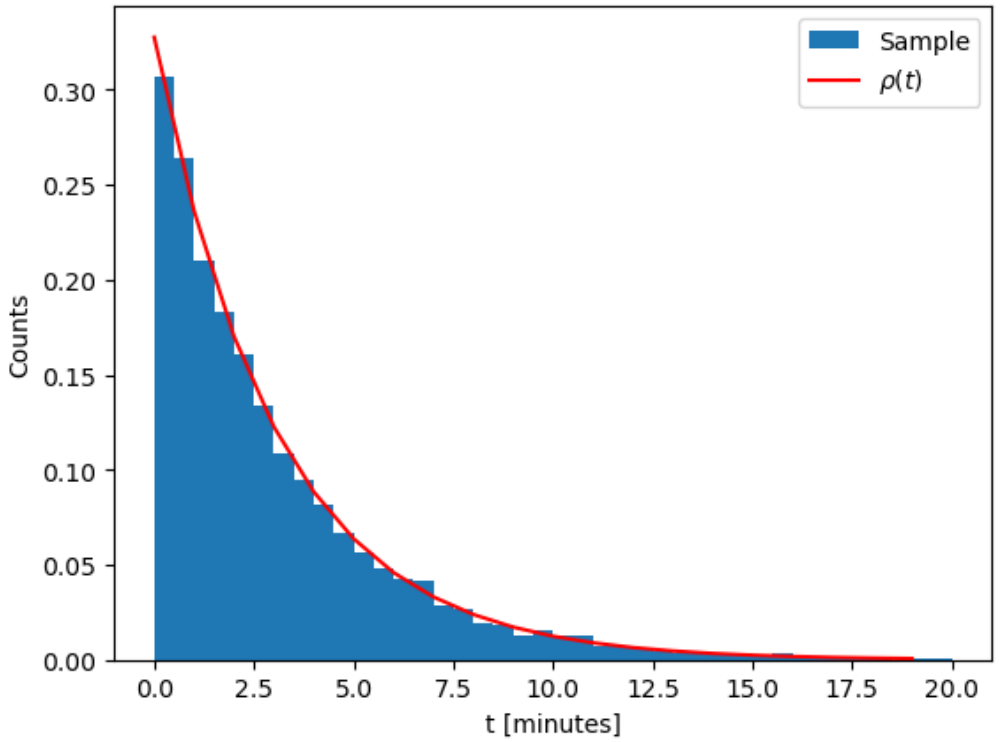
$$1 - e^{-\frac{t}{\tau}} = \eta.$$

4. Solving for t, we obtain:

$$t(\eta) = -\tau \ln(1 - \eta).$$

Sampling radioactive decay time

```python
## Radioactive decay sampler
def sample_tdecay(tau):
    eta = np.random.rand()
    return -tau * np.log(1-eta)


tau = 3.053 # Half-time in minutes
N = 10000   # Number of samples
tdecays = [sample_tdecay(tau) for i in range(N)]

# Show a histogram
plt.xlabel("t [minutes]")
plt.ylabel("Counts")
plt.hist(tdecays, bins = 40, range=(0,20), density=True)
```



Sampling points inside a circle

One way to sample points inside a unit circle is by switching to **polar coordinates**:

$$x = r\cos(\phi), \qquad y = r\sin(\phi),$$

where we sample r ∈ [0,1) and $\phi \in [0, 2\pi)$.

Naively, one could sample r and ϕ independently from two uniform distributions. Let's see what happens!

```
def sample_xy_naive():
    r = np.random.rand()
    phi = 2 * np.pi * np.random.rand()
    return r*np.cos(phi), r*np.sin(phi)

xplot = []
yplot = []
N = 1000
for i in range(N):
    x, y = sample_xy_naive()
    xplot.append(x)
    yplot.append(y)

plt.plot(xplot,yplot,'o',color='r')
plt.show()
```

Sampling points inside a circle

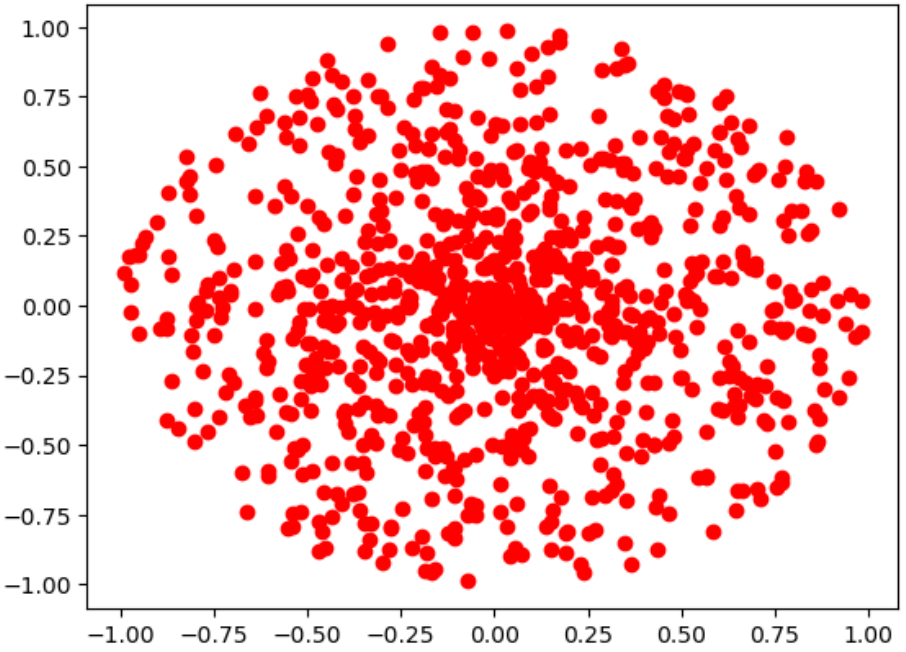
One way to sample points inside a unit circle is by switching to **polar coordinates**:

$$x = r\cos(\phi), \qquad y = r\sin(\phi),$$

where we sample r ∈ [0,1) and $\phi \in [0, 2\pi)$.

Naively, one could sample r and ϕ independently from two uniform distributions. Let's see what happens!

```
def sample_xy_naive():
    r = np.random.rand()
    phi = 2 * np.pi * np.random.rand()
    return r*np.cos(phi), r*np.sin(phi)

xplot = []
yplot = []
N = 1000
for i in range(N):
    x, y = sample_xy_naive()
    xplot.append(x)
    yplot.append(y)

plt.plot(xplot,yplot,'o',color='r')
plt.show()
```



The points clump more in the center!

Sampling points inside a circle

- The points **clump more in the center** because r is **not uniformly distributed**.
- Recall the **differential area element** in polar coordinates: $dxdy = rdrd\phi$
- This leads to the probability density functions: $\rho_r(r) = 2r, \qquad \rho_\phi(\phi) = \frac{1}{2\pi}.$
- **Cumulative Distribution Function (CDF)**

$$F_r(r) = \int_0^r \rho_r(r')dr' = r^2.$$

- To obtain a properly distributed r, we solve $F_r(r) = \eta$ which gives: $r = \sqrt{\eta}$

Sampling points inside a circle

- The points **clump more in the center** because r is **not uniformly distributed**.
- Recall the **differential area element** in polar coordinates: $dxdy = rdrd\phi$
- This leads to the probability density functions: $\rho_r(r) = 2r, \qquad \rho_\phi(\phi) = \frac{1}{2\pi}.$
- **Cumulative Distribution Function (CDF)**

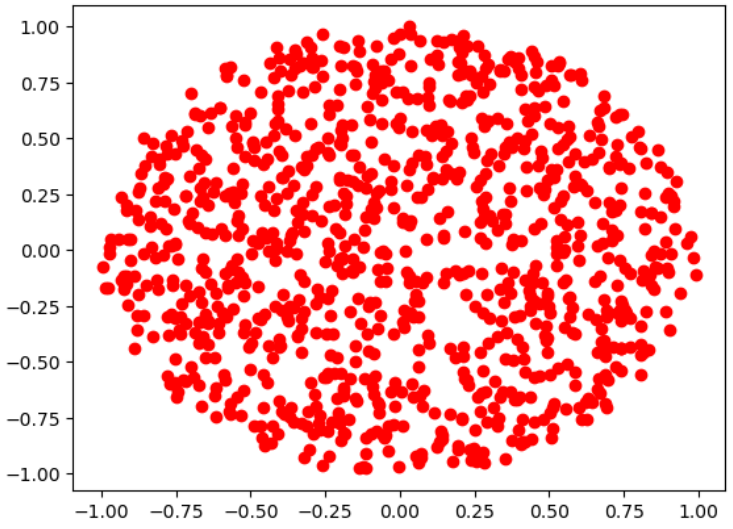
$$F_r(r) = \int_0^r \rho_r(r')dr' = r^2.$$

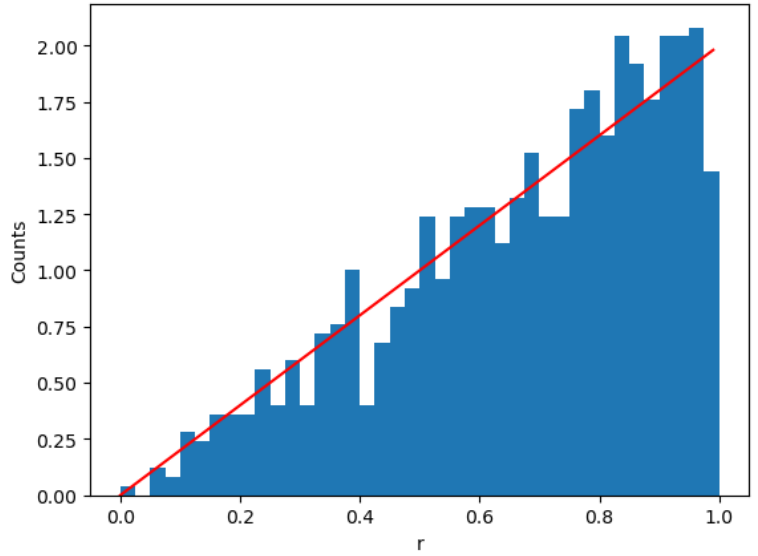
- To obtain a properly distributed r, we solve $F_r(r) = \eta$ which gives: $r = \sqrt{\eta}$

```python
def sample_xy_correct():
    eta = np.random.rand()
    r = np.sqrt(eta)
    phi = 2 * np.pi * np.random.rand()
    return r*np.cos(phi), r*np.sin(phi)
```

Other examples (extra slides):

- Isotropic direction (3D)
- Gaussian distribution





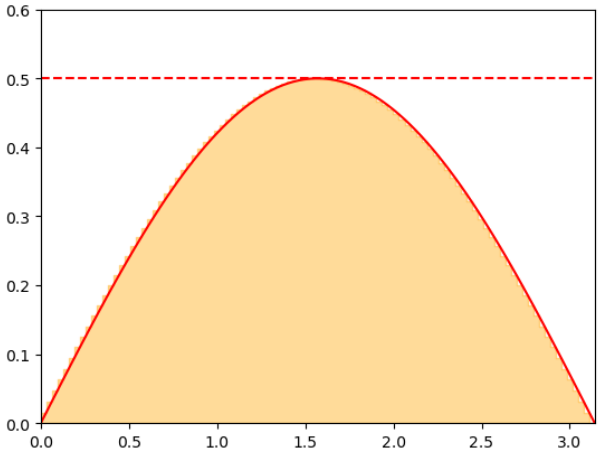
Rejection sampling

The **rejection sampling method** allows one to sample a variable ξ from an **envelope distribution** and accept or reject it with a certain probability.

Consider again the probability density function for the polar angle:

$$\rho_\theta(\theta) = \frac{\sin(\theta)}{2}.$$

Since ρ_θ is bounded from above, we define: $\rho_\theta^{\max} = 1/2.$



1. Sample a candidate value θ_{cand} from a uniform distribution over $(0, \pi)$.
2. Accept θ_{cand} with probability: $p = \rho_\theta(\theta_{\text{cand}})/\rho_\theta^{\max}$.
3. This step can be performed by sampling y **from a uniform distribution** over $(0, \rho_\theta^{max})$ and accepting θ_{cand} if $y < \rho_\theta(\theta_{cand})$

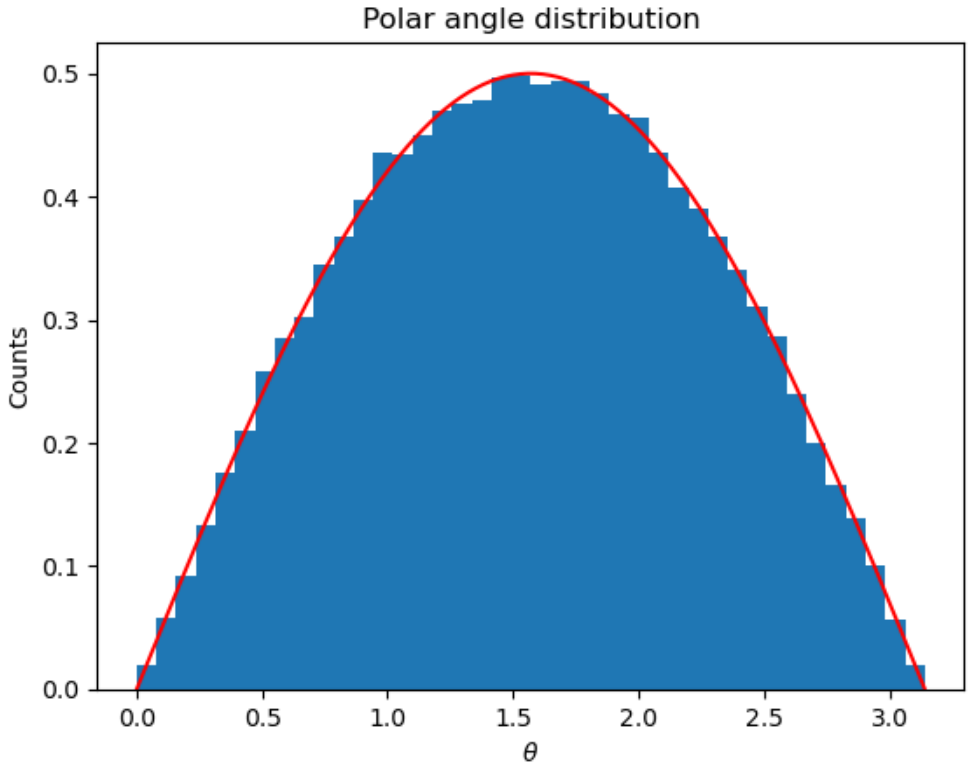
Geometric Interpretation: If we consider $\theta_{cand} = x$ and y as the **coordinates of a point** in a plane, we accept θ_{cand} **if it lies below the curve** defined by $\rho_\theta(\theta)$. This ensures that θ_{cand} values are accepted at a rate proportional to $\rho_\theta(\theta)$, as desired.

Advantages of Rejection Sampling:

$\rho_\theta(\theta)$ **does not need to be a normalized distribution** for the method to work.

Rejection sampling

```
def sample_rejection(rho, a, b, rhomax):
    while True:
        x_cand = a + (b-a)*np.random.rand()
        y = rhomax * np.random.rand()
        if (y < rho(x_cand)):
            return x_cand
    return 0.

def rho_theta(theta):
    return np.sin(theta) / 2.

N = 100000
samples = []
for i in np.arange(0,N,1):
    theta = sample_rejection(rho_theta, 0., np.pi, 0.5)
    samples.append(theta)
```



Pros and Cons of Rejection Sampling

Pros:

- Does not require the distribution to be normalized.
- Works even if y_{max} is **larger than the true maximum** of $\rho(x)$
- Applicable to **generic distributions** and does not require the evaluation of the cumulative distribution function.

Cons:

- Can be **inefficient** if the rejection rate is high (e.g., for highly peaked distributions).
- Not directly applicable to distributions **over infinite ranges**.

Generalizations of Rejection Sampling

To address some of its limitations, several generalizations of rejection sampling can be used, including:

- **Adaptive rejection sampling** by considering multiple **enveloping rectangles**.
- **Variable transformation** to map an **infinite interval** into a finite one.
- **Sampling from a non-uniform enveloping distribution** for better efficiency.

Importance sampling

Recall the calculation of an integral as statistical average

$$I = \int_a^b f(x)dx = (b-a)\langle f \rangle, \qquad \text{where} \qquad \langle f \rangle = \frac{1}{N}\sum_{i=1}^{N} f(x_i), \qquad x_i \in U(a,b)$$

Some issues with the method:

- Sample unimportant regions (e.g. f is highly peaked)
- Integrable singularities

Importance sampling:

Sample x_i from a *non-uniform* distribution w(x) that resembles f(x).

The integrand is then calculated as

$$I = \int_a^b \frac{f(x)}{w(x)} w(x)dx = \left\langle \frac{f(x)}{w(x)} \right\rangle_w$$

Normalization:

$$\int_a^b w(x)dx = 1.$$

Error:

$$\delta I = \frac{\sqrt{\left\langle \left[\frac{f(x)}{w(x)}\right]^2 \right\rangle_w - \left\langle \frac{f(x)}{w(x)} \right\rangle_w^2}}{\sqrt{N}}.$$

Importance sampling

$$I = \int_a^b \frac{f(x)}{w(x)} w(x)dx = \left\langle \frac{f(x)}{w(x)} \right\rangle_w \qquad \delta I = \frac{\sqrt{\left\langle \left[\frac{f(x)}{w(x)}\right]^2 \right\rangle_w - \left\langle \frac{f(x)}{w(x)} \right\rangle_w^2}}{\sqrt{N}}.$$

- For w(x)=1/(b-a) we recover the mean value method

$$I = \int_a^b f(x)dx = (b-a)\langle f \rangle$$

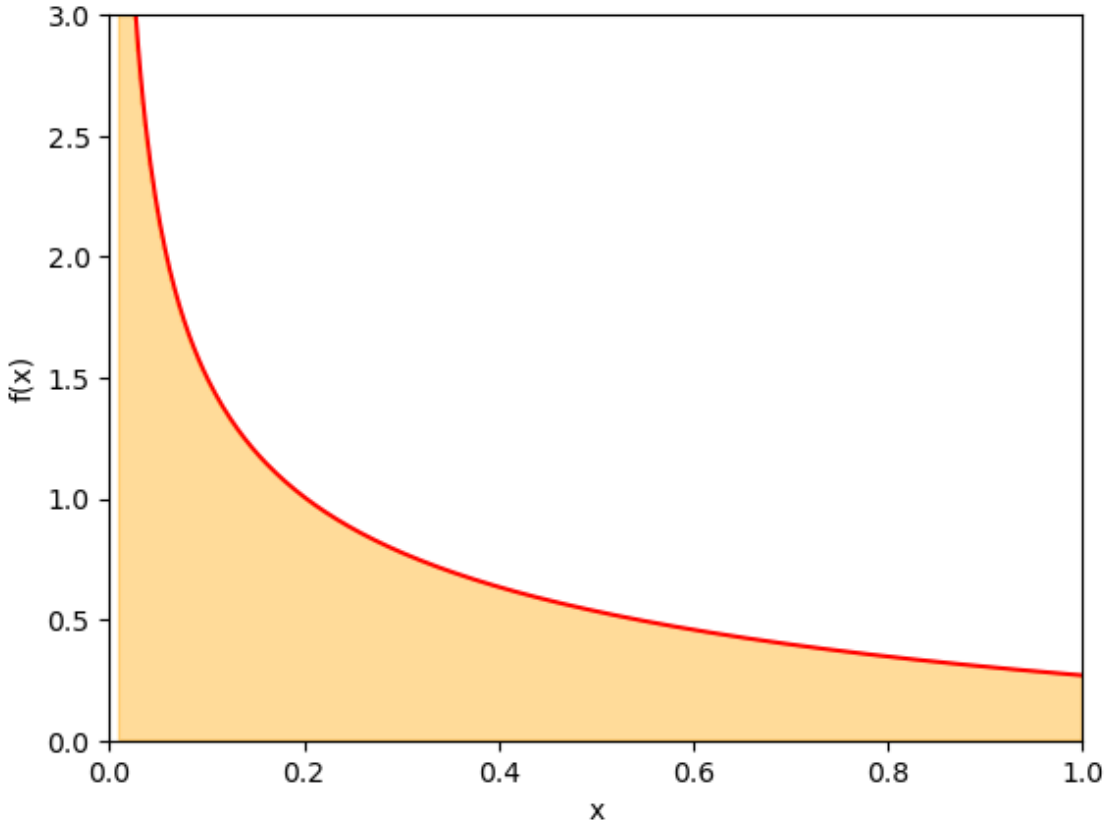
- For w(x) ∝ f(x) one has $\left\langle \frac{f(x)}{w(x)} \right\rangle = \text{const} = I$ and $\delta I = 0$

Importance sampling

```python
# Calculate integral \int_a^b f(x) dx using importance sampling
# f = f(x) is the integrand
# N is the number of random samples
# wx = w(x) is the normalized probability density from which
# the sampling takes place
# sampler is a function which samples a random number from w(x)
def intMC_weighted(f, N, wx, sampler):
    total = 0
    total_sq = 0
    for i in range(N):
        x = sampler()
        fval = f(x)
        total += fval / wx(x)
        total_sq += (fval / wx(x))**2
    fw_av = total / N
    fwsq_av = total_sq / N
    return fw_av, np.sqrt((fwsq_av - fw_av*fw_av)/N)
```

Importance sampling: Example

$$\int_0^1 \frac{x^{-1/2}}{e^x + 1} dx$$



Integrable singularity at x=0

Importance sampling: Example

$$\int_0^1 \frac{x^{-1/2}}{e^x + 1} dx$$

Mean value method

$$w(x) = \frac{1}{b-a}$$

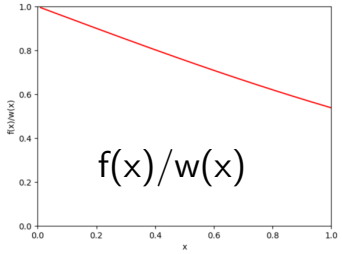
```
def uniform_sample():
    eta = np.random.rand()
    return eta

def uniform_w(x):
    return 1.

np.random.seed(1)
N = 1000000
I, err = intMC_weighted(f, N, uniform_w, uniform_sample)
print("I = ",I," +- ",err)
```

```
I =  0.8374063441946126  +-  0.0017772180714415427
```

Importance sampling

$$w(x) = \frac{1}{2\sqrt{x}} \qquad I = \left\langle \frac{2}{e^x + 1} \right\rangle_w \qquad x = \eta^2$$

```
def rsqrt_sample():
    eta = np.random.rand()
    return eta * eta

def rsqrt_w(x):
    return 1. / (2. * np.sqrt(x))
```




```
N = 1000000
I, err = intMC_weighted(f, N, rsqrt_w, rsqrt_sample)
print("I = ",I," +- ",err)
```

```
I =  0.839014917136739  +-  0.0001409071521618816
```

Statistical error is more than x10 smaller than in the mean value method.
We would need more than x100 samples in the mean value method to reach the same accuracy as importance sampling in this case.

Extra slides

Relaxation method

Relaxation method:

- Cast the equation f(x) = 0 in a form

$$x = \varphi(x)$$

- For example $\varphi(x) = f(x) + x$ but this choice is not unique

- The root is approximated by an iterative procedure

$$x_{n+1} = \varphi(x_n)$$

Convergence criterion:

$$|\varphi'(x_n)| < 1, \qquad \text{for all } x_n$$

Relaxation method

```
def relaxation_method(
    phi,                    # The function from the equation x = phi(x)
    x0,                     # The initial guess
    tolerance = 1.e-10,     # The desired accuracy of the solution
    max_iterations = 100    # Maximum number of iterations
    ):

    xprev = xnew = x0

    global last_relaxation_iterations
    last_relaxation_iterations = 0

    for i in range(max_iterations):
        last_relaxation_iterations += 1

        xprev = xnew
        xnew = phi(xprev) # The next iteration

        if (abs(xnew-xprev) < tolerance):
            return xnew


    print("The relaxation method failed to converge to a required precision in " + str(max_iterations) + " iterations")
    print("The error estimate is ", abs(xnew - xprev))

    return xnew
```

Relaxation method

$x + e^{-x} - 2 = 0$ as $x = 2 - e^{-x}$ i.e. $\phi(x) = 2 - e^{-x}$

Starting with x_0=0.5 we have

```
Solving the equation x = 2 - e^-x with relaxation method an initial guess of x0 =  0.5
Iteration:     0, x =    0.500000000000000, phi(x) = 1.393469340287367
Iteration:     1, x =    1.393469340287367, phi(x) = 1.751787325113973
Iteration:     2, x =    1.751787325113973, phi(x) = 1.826536369684999
Iteration:     3, x =    1.826536369684999, phi(x) = 1.839029855597129
Iteration:     4, x =    1.839029855597129, phi(x) = 1.841028423293983
Iteration:     5, x =    1.841028423293983, phi(x) = 1.841345821475382
Iteration:     6, x =    1.841345821475382, phi(x) = 1.841396170032424
Iteration:     7, x =    1.841396170032424, phi(x) = 1.841404155305379
Iteration:     8, x =    1.841404155305379, phi(x) = 1.841405421731432
Iteration:     9, x =    1.841405421731432, phi(x) = 1.841405622579610
Iteration:    10, x =    1.841405622579610, phi(x) = 1.841405654432999
Iteration:    11, x =    1.841405654432999, phi(x) = 1.841405659484766
Iteration:    12, x =    1.841405659484766, phi(x) = 1.841405660285948
Iteration:    13, x =    1.841405660285948, phi(x) = 1.841405660413011
Iteration:    14, x =    1.841405660413011, phi(x) = 1.841405660433162
Iteration:    15, x =    1.841405660433162, phi(x) = 1.841405660436358
The solution is x =  1.8414056604331623 obtained after  15  iterations
```

Not as fast as Newton-Raphson but does not require evaluation of the derivative

Relaxation method

$x^3 - x - 1 = 0$ as $x = x^3 - 1$ i.e. $\varphi(x) = x^3 - 1$

Starting with x_0=0.5 we have

```
Solving the equation x = x^3 - 1 with relaxation method an initial guess of x0 =  0.0
Iteration:     0, x =    0.000000000000000, phi(x) = -1.000000000000000
Iteration:     1, x =   -1.000000000000000, phi(x) = -2.000000000000000
Iteration:     2, x =   -2.000000000000000, phi(x) = -9.000000000000000
Iteration:     3, x =   -9.000000000000000, phi(x) = -730.000000000000000
Iteration:     4, x = -730.000000000000000, phi(x) = -389017001.000000000000000
Iteration:     5, x = -389017001.000000000000000, phi(x) = -588715871622705914576896000.000000000000000
Iteration:     6, x = -588715871622705914576896000.000000000000000, phi(x) = -204040901322752646989478259680513109526757826056202557355691431285390611316736
62025573556914312853906113167360.000000000000000
Iteration:     7, x = -204040901322752646989478259680513109526757826056202557355691431285390611316736.000000000000000, phi
(x) = -84947714722373876912426115385994721993330450340708886432958705831500286122585831451013021195433672849326160977228141311271042752909937066699439435575188250417201392567517562965143635104635017828056961674070967914149432730331633418240.00000000
0000000
```

Divergent!

Reason: $|\varphi'(x_n)| < 1$ violated [try to come up with a better form of $\varphi(x)$?]

Sampling of an Isotropic Direction in 3D

One common problem in **Monte Carlo simulations** is the **random sampling of an isotropic direction in 3D space**. This issue arises in various contexts, such as:

- Sampling a random orientation of an **axially symmetric object** (e.g., a rod).
- Sampling the **momentum direction** of a particle.

This problem is equivalent to **choosing a random point on a unit sphere**. The coordinates x, y, z on the sphere can be parametrized using **azimuthal** and **polar angles**:

- $\phi \in [0, 2\pi)$ (azimuthal angle)
- $\theta \in [0, \pi)$ (polar angle)

Using these angles, the Cartesian coordinates are given by:

$$x = \sin\theta \cos\phi$$

$$y = \sin\theta \sin\phi$$

$$z = \cos\theta$$

Sampling of an Isotropic Direction in 3D

$$x = \sin\theta\cos\phi$$
$$y = \sin\theta\sin\phi$$
$$z = \cos\theta$$

- The solid angle element is:

$$d\Omega = \sin(\theta)d\theta d\phi,$$

- The random variables ϕ and θ are independent.
- ϕ is uniformly distributed in $[0, 2\pi]$, making its sampling straightforward.
- However, θ has a **weighted probability density function**: $\rho_\theta(\theta) = \frac{1}{2}\sin(\theta),$

The cumulative distribution function is:

$$F_\theta(\theta) = \int_0^\theta \frac{1}{2}\sin(\theta')d\theta' = \frac{1 - \cos(\theta)}{2}$$

Solving $F_\theta(\theta) = \eta$, we obtain:

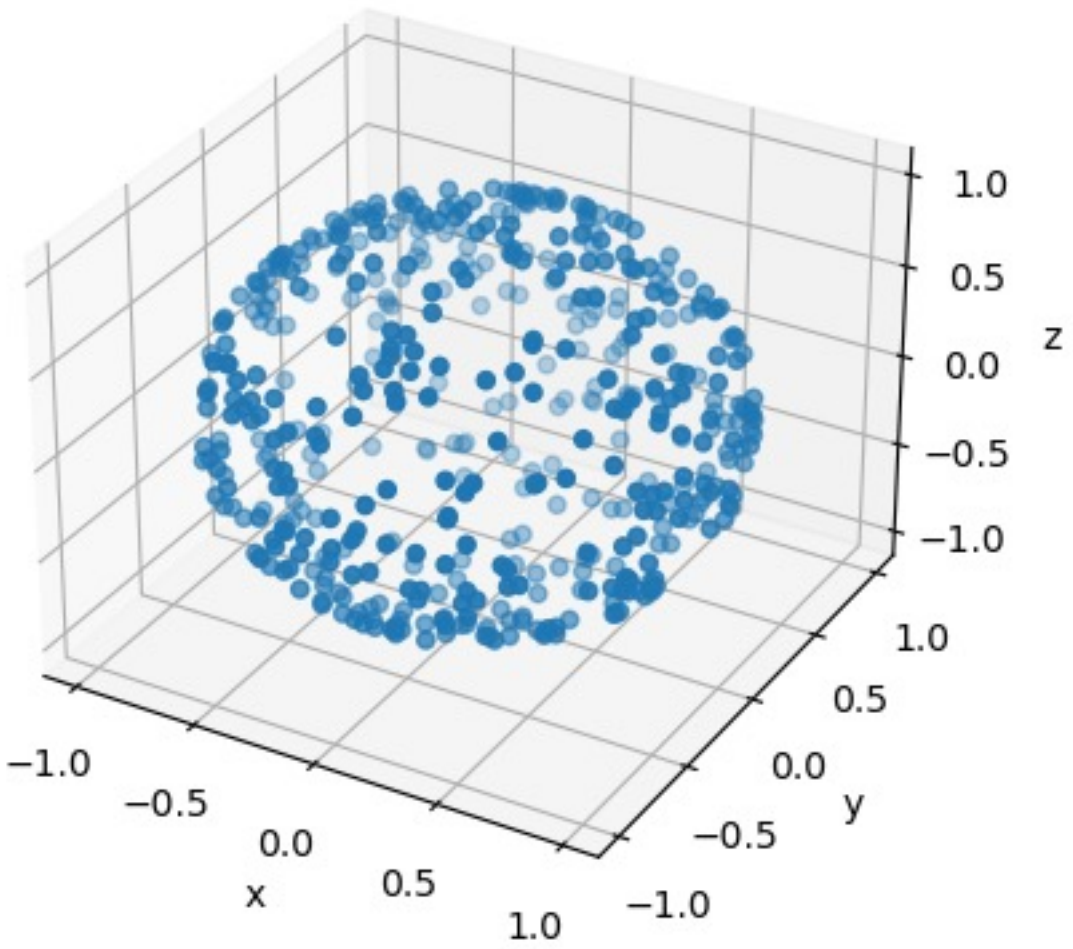
$$\theta = \arccos(2\eta - 1).$$

In practice, work directly with $\cos\theta$ and $\sin\theta$:

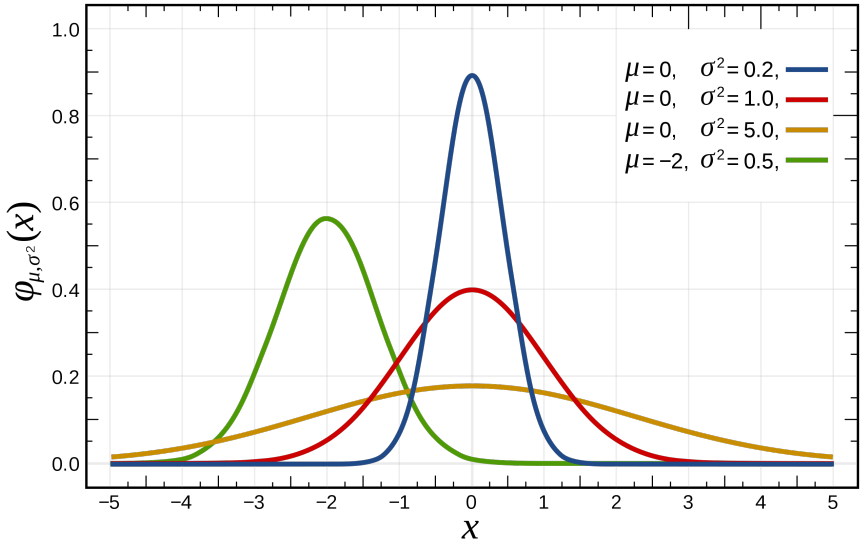
$$\cos(\theta) = 2\eta - 1, \qquad \sin(\theta) = \sqrt{1 - [\cos(\theta)]^2}$$

Sampling an isotropic direction

```python
def sample_xyz_isotropic():
    phi = 2 * np.pi * np.random.rand()
    costh = 2 * np.random.rand() - 1
    sinth = np.sqrt(1-costh*costh)
    return sinth * np.cos(phi), sinth * np.sin(phi), costh

xplot = []
yplot = []
zplot = []
N = 500
for i in range(N):
    x, y, z = sample_xyz_isotropic()
    xplot.append(x)
    yplot.append(y)
    zplot.append(z)

fig = plt.figure()
ax = fig.add_subplot(projection='3d')

ax.scatter(xplot,yplot,zplot)

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')

plt.show()
```



Sampling normally distributed variables



One of the most common probability distributions is the **normal (or Gaussian) distribution**, given by:

$$\rho(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

There are many standard methods for sampling from this distribution. One common approach is to **standardize the variable** by making the transformation $x \to \mu + \sigma x$

After this transformation, the new variable follows a **standard normal distribution** with **zero mean** and **unit standard deviation**:

$$\rho(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}.$$

Calculating the cumulative distribution function $F(x) = \int_{-\infty}^{x} \rho(x')dx'$ is **not straightforward**, use numerical methods.

Sampling normally distributed variables

Instead of one variable, we can consider a pair of independent normally distributed variables x, y:

$$\rho(x, y) = \frac{1}{2\pi} e^{-\frac{x^2}{2}} e^{-\frac{y^2}{2}},$$

Making a change of variables to polar coordinates

$$x = r\cos(\phi), \qquad y = r\sin(\phi),$$

and taking into account

$$dxdy = rdrd\phi$$

we get

$$\rho(r, \phi) = \frac{1}{2\pi} re^{-r^2/2}.$$

Therefore, we can sample x and y by sampling two independent random variables r and ϕ. ϕ is uniformly distributed in $[0, 2\pi)$. For r we have the following probability density

$$\rho_r(r) = re^{-r^2/2},$$

and the cumulative distribution function

$$F_r(r) = \int_0^r r' e^{-r'^2/2} dr' = 1 - e^{-r^2/2},$$

therefore

$$r = \sqrt{-2\ln(1-\eta)}.$$

Sampling normally distributed variables

```python
def sample_xy_normal():
    phi = 2 * np.pi * np.random.rand()
    eta = np.random.rand()
    r = np.sqrt(-2*np.log(1-eta))
    return r * np.cos(phi), r * np.sin(phi)

N = 10000
samples = []
for i in np.arange(0,N,2):
    x, y = sample_xy_normal()
    samples.append(x)
    samples.append(y)
```